

Dossier Technique – Concours InnovAM'25



Réalisé par :

- Aymane Elhaoudar
- Mohamed Amine Mohib
- Yassine Benkhoulouk
- Nassim Bouziki

Sommaire

- 01 Introduction
 - Contexte du projet
 - Objectif du robot d'inspection
 - Description du système robotique proposé
- 02 Problématique
 - Contexte de l'inspection des rames TGV
 - Complexité et risques associés à l'inspection manuelle
 - Solution robotisée : avantages et objectifs
- 03 Analyse fonctionnelle
 - Description du diagramme pieuvre
 - Objectifs et fonctions principales du robot d'inspection
- 04 Étude mécanique
 - Conception et dimensionnement du bras robotisé
 - Mécanismes et moteurs utilisés pour le contrôle
- 05 Système de navigation
 - Utilisation de la caméra Raspberry Pi V3 pour le suivi d'opérateur
 - Algorithmes de détection d'obstacles avec OpenCV
 - Système de contrôle avec l'unité esclave Teensy 4.1
- 06 Capteurs et unités embarquées
 - Unité Maître (Raspberry Pi 5) et ses capacités
 - Capteurs LiDAR et leurs rôles pour la cartographie 3D
 - Module LoRa pour la communication longue portée
 - Capteurs ultrason et de température pour la sécurité

07

Choix de microcontrôleur

- Sélection et justification du microcontrôleur ESP32
- Avantages de l'ESP32 pour les applications de robotique

08

Dimensionnement du Raspberry Pi 5

- Utilisation et dimensionnement du Raspberry Pi 5 dans le système
- Consommation d'énergie et autonomie

09

Caméra et imagerie

- Spécifications de la caméra utilisée (Module Camera 3)
- Fonctionnalités de capture et traitement d'images

10

Fonctionnement du LiDAR

- Utilisation du LiDAR pour la navigation autonome
- Cartographie 3D et détection d'obstacles

11

Commandes et système de contrôle

- Système de commande autonome et manuel
- Architecture globale du système de commande
- Communication entre les différents modules du robot

12

Système de surveillance et tableau de bord

- Surveillance des anomalies avec InfluxDB
- Installation et configuration du tableau de bord
- Fonctionnalités de détection et analyse en temps réel des anomalies

- 13 Dimensionnement et alimentation *p2*
- Dimensionnement du moteur et de la batterie pour le robot
 - Considérations d'autonomie et consommation d'énergie
- 14 Conception et câblage *p4*
- Schémas de câblage du contrôle robotisé
 - Conception du PCB pour le contrôle du robot
- 15 Bibliographie *p5*
- Sources et références utilisées dans le développement du projet

Dossier Technique – Concours InnovAM'25

1. Introduction

Le projet proposé par **Siana** consiste en la conception et la réalisation d'un robot d'inspection de rames TGV, intégrant des technologies avancées en vision artificielle, navigation autonome et communication en temps réel. Ce dossier technique détaille les solutions retenues pour la réalisation du prototype.

SIANA : acteur de l'Ecosystème Ferroviaire et bâtisseur de l'ambition de la mobilité de demain.

Agissant en moteur de la transformation de la mobilité au Maroc, SIANA est une filiale commune de deux acteurs de référence de l'exploitation ferroviaire, l'ONCF et SNCF.

« Au-delà de la réponse au défi quotidien de transporter des milliers de Clients à 320 km/h entre TANGER et CASABLANCA, l'objectif de SIANA et de ses collaborateurs, pionniers de la Très Grande Vitesse au Maroc, est d'offrir une mobilité propre et efficace aux Dynamiques et Ambitieux qui pensent à l'environnement qu'ils laisseront en héritage à leurs petits-enfants!

Pour un monde meilleur »

Luciano FERNANDES BORGES
DIRECTEUR GENERAL



L'Excellence

La Transparence

La Rigueur

L'Engagement

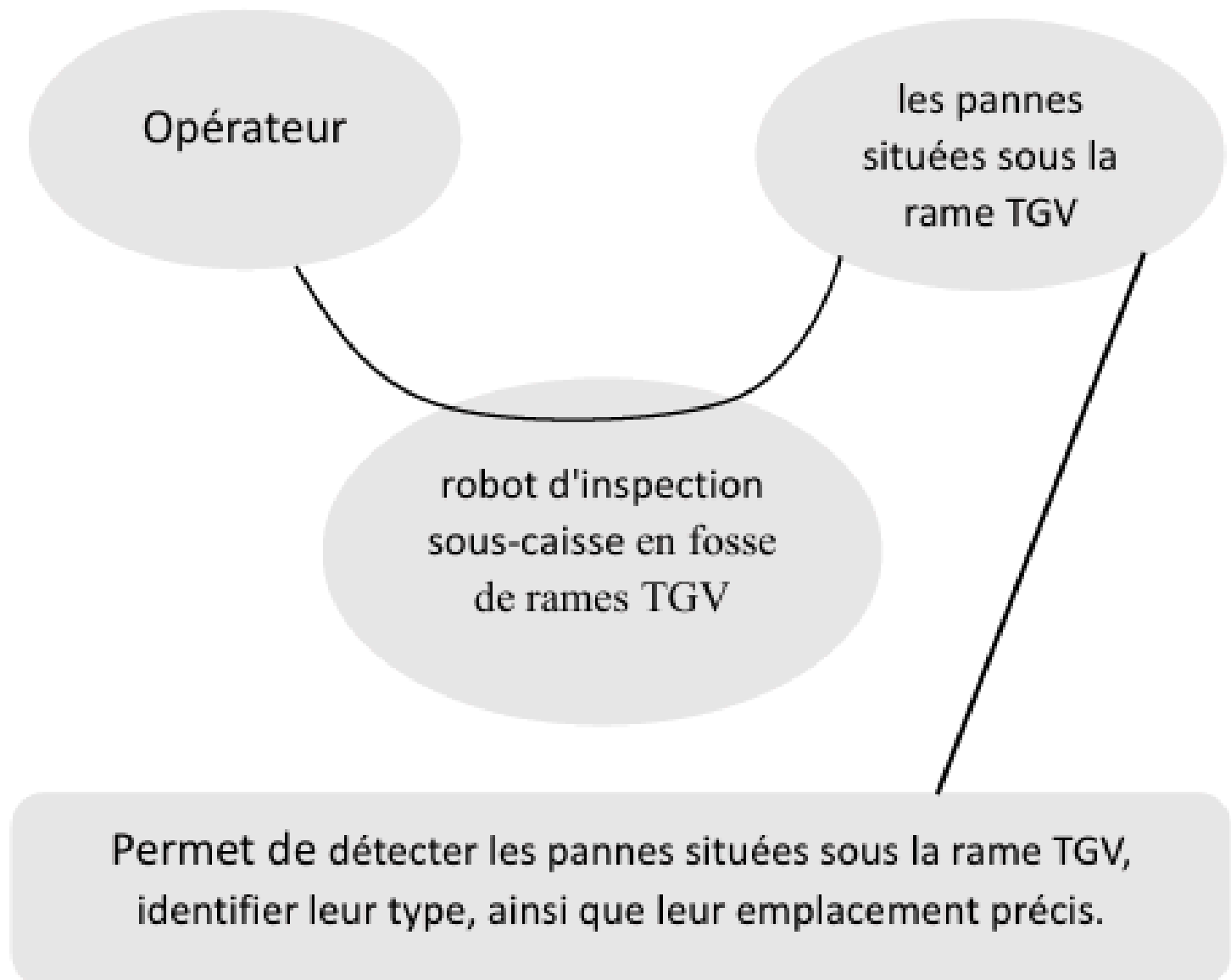
L'Excellence

Pour viser le meilleur, stimuler l'innovation, la créativité et challenger les meilleures pratiques dans le domaine ferroviaire

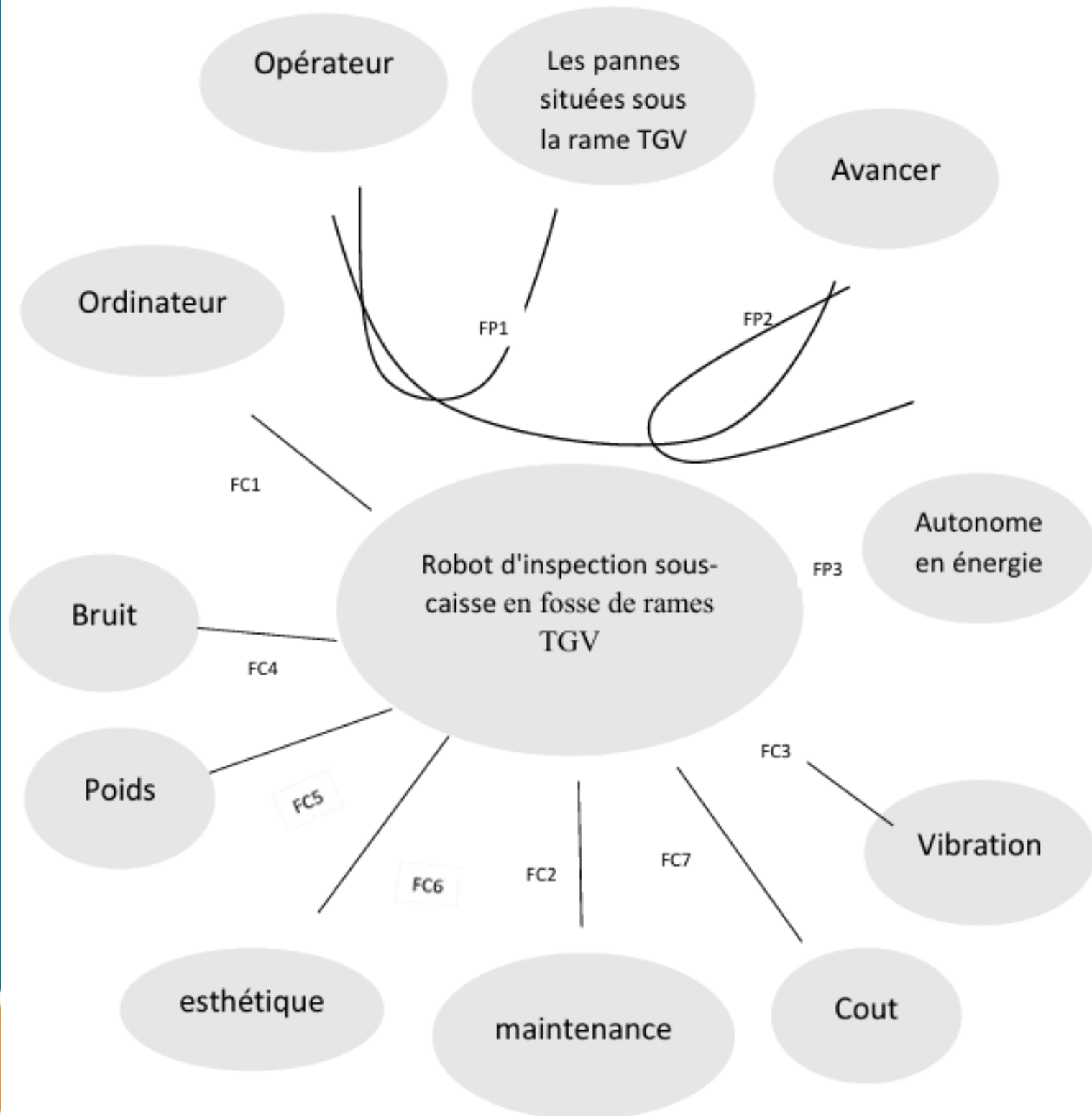
1.2. Problématique

L'inspection des rames TGV en fosse de maintenance est une tâche cruciale mais souvent complexe et risquée lorsqu'elle est réalisée manuellement. Afin de rendre cette opération plus rapide, sûre et efficace, nous souhaitons développer un robot autonome capable d'inspecter le dessous des rames, de repérer les pannes et de les localiser précisément pour faciliter les réparations.

- **Analyse fonctionnelle**



- Le diagramme pieuvre



Fonction	Description
<u>FP1</u>	Permet de détecter les pannes situées sous la rame TGV, identifier leur type, ainsi que leur emplacement précis.
<u>FP2</u>	Permet de translater linéairement le Robot.
<u>FP3</u>	Fonctionner de manière autonome en utilisant sa propre source d'énergie, sans nécessiter de recharge fréquente
<u>FC1</u>	
<u>FC2</u>	Facilité l'entretien et la durabilité du système.
<u>FC3</u>	Conserver le mouvement et la stabilité du robot.
<u>FC4</u>	Fonctionner d'une manière silencieuse.
<u>FC5</u>	Être manipulable si besoin de le déplacer par au plus 2 personnes en termes de poids.
<u>FC6</u>	Design respectueux de l'esthétique.
<u>FC7</u>	Cout financièrement accessible.

Étude mécanique

conception et dimensionnement du bras

Modèle géométrique direct du robot rigide :

Définition :

Le modèle géométrique directe (MGD) est l'ensemble des relations qui permettent d'exprimer la situation de l'organe terminal. Le MGD exprime la position et l'orientation du repère de référence R lié à l'outil, relativement à un repère fixe celui de la base R0 en fonction des variables articulaires motorisée du mécanisme. L'élément fondamental qui décrit la position et l'orientation d'un repère Ri, d'origine Oi, dans un repère Rj peut se mettre sous une forme matricielle.

Obtenir le modèle géométrique direct consiste à déterminer les relations de la forme suivante :

$$X = f(\theta)$$

θ est de dimension n, X est de dimension m, Permettant de calculer X, le vecteur des coordonnées choisies pour représenter la position de l'organe terminal, par rapport au repère R0, à partir de la connaissance de θ , et le vecteur des coordonnées articulaires

Configuration et situation d'un bras manipulateur :

De manière classique, la configuration d'un système mécanique est connue quand la position de tous ses points dans R0 est connue. Pour un bras manipulateur, elle est définie par un vecteur θ de n coordonnées indépendante appelées coordonnées généralisées et elle est égale au nombre d'articulations motorisées.

La configuration est alors définie sur un espace N dont la dimension n est appelée indice de mobilité. N est appelé espace de configuration. Les coordonnées généralisées correspondent aux grandeurs caractéristiques des différentes articulations :

On note $\theta = \theta_1, \theta_2, \dots, \theta_n$

La situation de X de l'organe terminal est alors définie par m coordonnées indépendantes dites coordonnées opérationnelles, qui donnent la position et l'orientation de l'organe terminal dans R0. On définira généralement la situation en fonction de la tâche à accomplir : par exemple on pourra considérer que la position de l'organe terminal et non plus son orientation dans tous les cas, la situation de l'organe terminal est définie sur un espace M, de dimension $m \leq 6$, appelé espace opérationnel.

On note : $x(x_1, x_2, \dots, x_m)^T$

Le choix le plus commun pour les paramètres de position est celui des coordonnées cartésiennes. L'expression de l'orientation de l'organe terminal dépend du choix des paramètres angulaires.

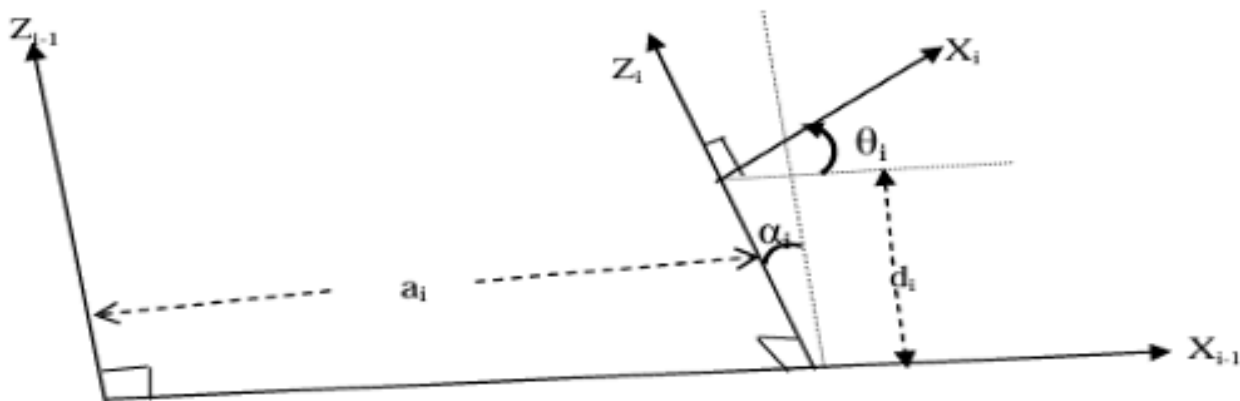
Pour la modélisation des robots il faut une méthode adéquate pour la description de leur morphologie. Plusieurs méthodes ont été proposées: [Renaud 75], [Khaliil 76], [Borrel 79], la méthode la plus répandue est Denavit-Hartenberg [Denavit 55], cette méthode est développée pour des structures ouvertes simples, et dans notre cas c'est cette méthode qu'on va utiliser.

Paramètres de Denavit-Hartenberg :

Denavit et Hartenberg ont développé en 1955 une méthode systématique basée sur la matrice de transformation homogène. Elle permet le passage entre articulations adjacentes d'un système robotique. Elle concerne les chaînes cinématiques ouvertes où l'articulation possède uniquement un degré de liberté, et les surfaces adjacentes restent en contact. Pour déterminer les paramètres géométriques du robot, on utilise la méthode de Denavit et Hartenberg qui impose les conventions suivantes :

- Le robot est composé de six corps noté : $c_0, \dots, c_5$ avec c_0 désigne la base du robot et c_5 désigne le corps porteur de l'organe terminal.
- L'articulation i connecte le corps c_{i-1} au corps c_i .

- Les corps sont rigides (les articulations du robot sont supposées idéales).
- La variable de l'articulation est notée θ_i .
- On associe au corps ci le repère R_i qui est défini comme suit : -L'axe Z_i est confondu avec l'axe de l'articulation i .
- -L'axe X_{i-1} est porté par la perpendiculaire commune aux axes Z_{i-1} et Z_i .
- -Si les axes Z_{i-1} et Z_i sont parallèles ou colinéaires le choix de l'axe X_i se fait de telle manière à simplifier notre système rendre di nul.
- -Si Z_{i-1} et Z_i ne se coupent pas et on choisit X_i de manière à être la parallèle avec l'axe perpendiculaire à Z_{i-1} et Z_i .
- On place les axes Z_i portés par les axes des articulations puis les axes X_i .
- Le passage d'un repère R_{i-1} au repère R_i est défini par quatre paramètres : - α_i angle entre Z_{i-1} et Z_i (rotation autour de X_i).
- - a_i distance entre Z_{i-1} et Z_i le long de X_i .
- - θ_i angle entre X_{i-1} et X_i (rotation autour de Z_i).
- - d_i distance entre X_{i-1} et X_i le long de Z_i .



La matrice de transformation homogène s'écrit donc :

$${}^{i-1}T_i = \text{Rot}(Z_{i-1}, \theta_i) \cdot \text{Trans}(Z_{i-1}, d_i) \cdot \text{Trans}(X_i, a_i) \cdot \text{Rot}(X_i, \alpha_i)$$

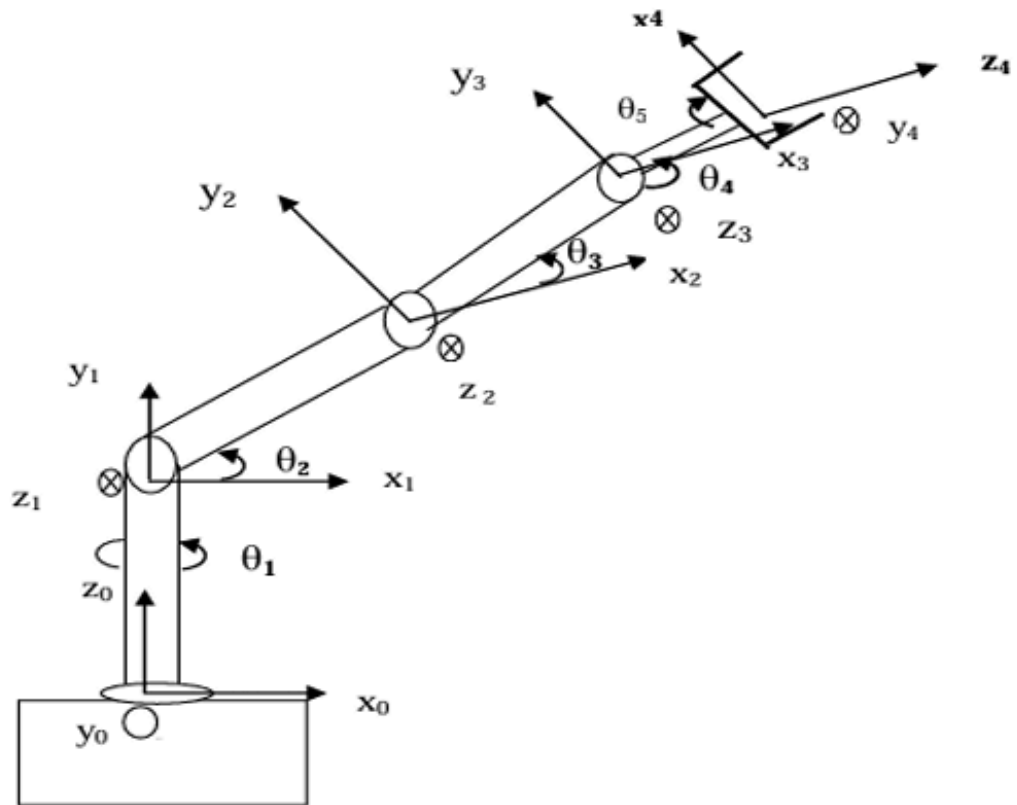
$${}^{i-1}T_i =$$

$$\begin{bmatrix} \cos \theta_i & -\sin \theta_i & 0 & 0 \\ \sin \theta_i & \cos \theta_i & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}
 \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}
 \begin{bmatrix} 1 & 0 & 0 & a_i \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}
 \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \alpha_i & -\sin \alpha_i & 0 \\ 0 & \sin \alpha_i & \cos \alpha_i & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

La matrice de transformation homogène s'écrit donc :

$${}^{i-1}T_i = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \alpha_i \cos \theta_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

La figure montre les paramètres intervenant dans la transformation :



	θ_1	a_1	d_1	α_1
Segment n°1	θ_1	0	L_1	$\pi/2$
Segment n°2	θ_2	L_2	0	0
Segment n°3	θ_3	L_3	0	0
Segment n°4	θ_4	0	0	$\pi/2$
Segment n°5	θ_5	0	L_4	0

Calcul du modèle géométrique direct :

Calcul de matrice de position:

$${}^0T_1 = \text{Rot}(Z_0, \theta_1) \cdot \text{Trans}(Z_0, d_1) \cdot \text{Trans}(X_1, a_1) \cdot \text{Rot}(X_1, \alpha_1)$$

$${}^0T_1 = \begin{bmatrix} \cos \theta_1 & 0 & \sin \theta_1 & 0 \\ \sin \theta_1 & 0 & -\cos \theta_1 & 0 \\ 0 & 1 & 0 & L_1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$${}^1T_2 = \text{Rot}(Z_1, \theta_2) \cdot \text{Trans}(Z_1, d_2) \cdot \text{Trans}(X_2, a_2) \cdot \text{Rot}(X_2, \alpha_2)$$

$${}^1T_2 = \begin{bmatrix} \cos \theta_2 & -\sin \theta_2 & 0 & L_2 \cos \theta_2 \\ \sin \theta_2 & \cos \theta_2 & 0 & L_2 \sin \theta_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$${}^2T_3 = \text{Rot}(Z_2, \theta_3) \cdot \text{Trans}(Z_2, d_3) \cdot \text{Trans}(X_3, a_3) \cdot \text{Rot}(X_3, \alpha_3)$$

$${}^2T_3 = \begin{bmatrix} \cos \theta_3 & -\sin \theta_3 & 0 & L_3 \cos \theta_3 \\ \sin \theta_3 & \cos \theta_3 & 0 & L_3 \sin \theta_3 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$${}^0T_3 = {}^0T_1 \cdot {}^1T_2 \cdot {}^2T_3$$

$${}^0T_3 = \begin{bmatrix} \cos \theta_1 \cos (\theta_2 + \theta_3) & -\cos \theta_1 \sin (\theta_2 + \theta_3) & \sin \theta_1 & \cos \theta_1 (L_2 \cos \theta_2 + L_3 \cos (\theta_2 + \theta_3)) \\ \sin \theta_1 \cos (\theta_2 + \theta_3) & -\sin \theta_1 \sin (\theta_2 + \theta_3) & -\cos \theta_1 & \sin \theta_1 (L_2 \cos \theta_2 + L_3 \cos (\theta_2 + \theta_3)) \\ \sin (\theta_2 + \theta_3) & \cos (\theta_2 + \theta_3) & 0 & L_1 + L_2 \sin \theta_2 + L_3 \sin (\theta_2 + \theta_3) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Le vecteur de position est :

$$\text{Tr} = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix} = \begin{bmatrix} \cos \theta_1 (L_2 \cos \theta_2 + L_3 \cos (\theta_2 + \theta_3)) \\ \sin \theta_1 (L_2 \cos \theta_2 + L_3 \cos (\theta_2 + \theta_3)) \\ L_1 + L_2 \sin \theta_2 + L_3 \sin (\theta_2 + \theta_3) \end{bmatrix}$$

Calcul de matrice de rotation :

$${}^3T_4 = \text{Rot}(Z_3, \theta_4) \cdot \text{Trans}(Z_3, d_4) \cdot \text{Trans}(X_4, a_4) \cdot \text{Rot}(X_4, \alpha_4)$$

$${}^3T_4 = \begin{bmatrix} \cos \theta_4 & 0 & \sin \theta_4 & 0 \\ \sin \theta_4 & 0 & -\cos \theta_4 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$${}^4T_5 = \text{Rot}(Z_4, \theta_5) \cdot \text{Trans}(Z_4, d_5) \cdot \text{Trans}(X_5, a_5) \cdot \text{Rot}(X_5, \alpha_5)$$

$${}^4T_5 = \begin{bmatrix} \cos \theta_5 & -\sin \theta_5 & 0 & 0 \\ \sin \theta_5 & \cos \theta_5 & 0 & 0 \\ 0 & 0 & 1 & L_4 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$${}^3T_5 = {}^3T_4 \cdot {}^4T_5$$

$${}^3T_5 = \begin{bmatrix} \cos \theta_4 \cos \theta_5 & -\cos \theta_4 \sin \theta_5 & \sin \theta_4 & L_4 \sin \theta_4 \\ \sin \theta_4 \cos \theta_5 & -\sin \theta_4 \sin \theta_5 & -\cos \theta_4 & -L_4 \cos \theta_4 \\ \sin \theta_5 & \cos \theta_5 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

La matrice de rotation est donc :

$$R = {}^3T_5 = \begin{bmatrix} \cos \theta_4 \cos \theta_5 & -\cos \theta_4 \sin \theta_5 & \sin \theta_4 \\ \sin \theta_4 \cos \theta_5 & -\sin \theta_4 \sin \theta_5 & -\cos \theta_4 \\ \sin \theta_5 & \cos \theta_5 & 0 \end{bmatrix}$$

Par identification avec la matrice de rotation des angles de tangage roulis et lacet qui est donné par :

$$R = \begin{bmatrix} \cos \alpha \cos \beta & -\sin \alpha \cos \gamma + \sin \beta \cos \alpha \sin \gamma & \sin \alpha \sin \gamma + \cos \alpha \sin \beta \sin \gamma \\ \sin \alpha \cos \beta & \cos \alpha \cos \gamma + \sin \alpha \sin \beta \sin \gamma & -\cos \alpha \sin \gamma + \sin \beta \sin \alpha \cos \gamma \\ -\sin \beta & \sin \gamma \cos \beta & \cos \beta \cos \gamma \end{bmatrix}$$

On obtient alors :

$$\cos \theta_4 \cos \theta_5 = \cos \alpha \cos \beta = a_x$$

$$\sin \theta_4 \cos \theta_5 = \sin \alpha \cos \beta = a_y$$

$$\sin \theta_5 = -\sin \beta = a_z$$

$$\cos \theta_5 = \sin \gamma \cos \beta = b_z$$

$$\cos \beta \cos \gamma = 0 = c_z$$

On a: $\beta \neq \pi/2$:

Donc:

$$\alpha = \text{atan2}(a_y, a_x) \quad \text{d'où } \alpha = \theta_4$$

$$\beta = \text{atan2}(-a_z, \sqrt{a_x^2 + a_y^2}) \quad \text{d'où } \beta = \theta_5$$

$$\gamma = \text{atan2}(b_z, c_z) \quad \text{d'où } \gamma = \pi/2$$

Modèle géométrique inverse du robot rigide :

Définition :

Le problème consiste à calculer les coordonnées articulaires du robot rigide à partir des coordonnées opérationnelles.

Solubilité du MGI :

La solubilité du MGI, n'est autre que l'existence d'un nombre fini de solutions fondamental en matière de conception. Supposons que la situation x d'un bras manipulateur à n liaisons est exprimée par un nombre m minimal de paramètres. Supposons par ailleurs que x est une situation accessible avec le bras manipulateur, c'est-à-dire que la situation appartient à l'espace de travail. Alors :

- Si $n < m$ il n'existe pas de solution pour le MGI.
- si $n = m$ il existe un nombre fini de solutions en dehors de certaines configurations, appelées singulières.
- Si $n > m$, il existe une infinité de solutions.

Pour calculer le MGI, on a le choix entre plusieurs méthodes, on cite, la méthode de Paul, la méthode de Pieper et la méthode générale de Raghavan et Roth. En ce qui nous concerne nous avons choisi de séparer la position du bras de celle de l'organe terminal.

Calcul du modèle géométrique inverse :

Le calcul du modèle inverse en position est fait par un autre binôme.

Calcul de la matrice en orientation (de l'organe terminal) :

$${}^3T_5 = {}^3T_4 \cdot {}^4T_5$$

$${}^3T_5 = \begin{bmatrix} \cos \theta_4 \cos \theta_5 & -\cos \theta_4 \sin \theta_5 & \sin \theta_4 & L_4 \sin \theta_4 \\ \sin \theta_4 \cos \theta_5 & -\sin \theta_4 \sin \theta_5 & -\cos \theta_4 & -L_4 \cos \theta_4 \\ \sin \theta_5 & \cos \theta_5 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

La matrice de rotation est donc :

R=

$$\begin{bmatrix} \cos \alpha \cos \beta & -\sin \alpha \cos \gamma + \sin \beta \cos \alpha \sin \gamma & \sin \alpha \sin \gamma + \cos \alpha \sin \beta \sin \gamma \\ \sin \alpha \cos \beta & \cos \alpha \cos \gamma + \sin \alpha \sin \beta \sin \gamma & -\cos \alpha \sin \gamma + \sin \beta \sin \alpha \cos \gamma \\ -\sin \beta & \sin \gamma \cos \beta & \cos \beta \cos \gamma \end{bmatrix}$$

On obtient alors :

$$\cos \theta_4 \cos \theta_5 = \cos \alpha \cos \beta = a_x$$

$$\sin \theta_4 \cos \theta_5 = \sin \alpha \cos \beta = a_y$$

$$\sin \theta_5 = -\sin \beta = a_z$$

$$\cos \theta_5 = \sin \gamma \cos \beta = b_z$$

$$\cos \beta \cos \gamma = 0 = c_z$$

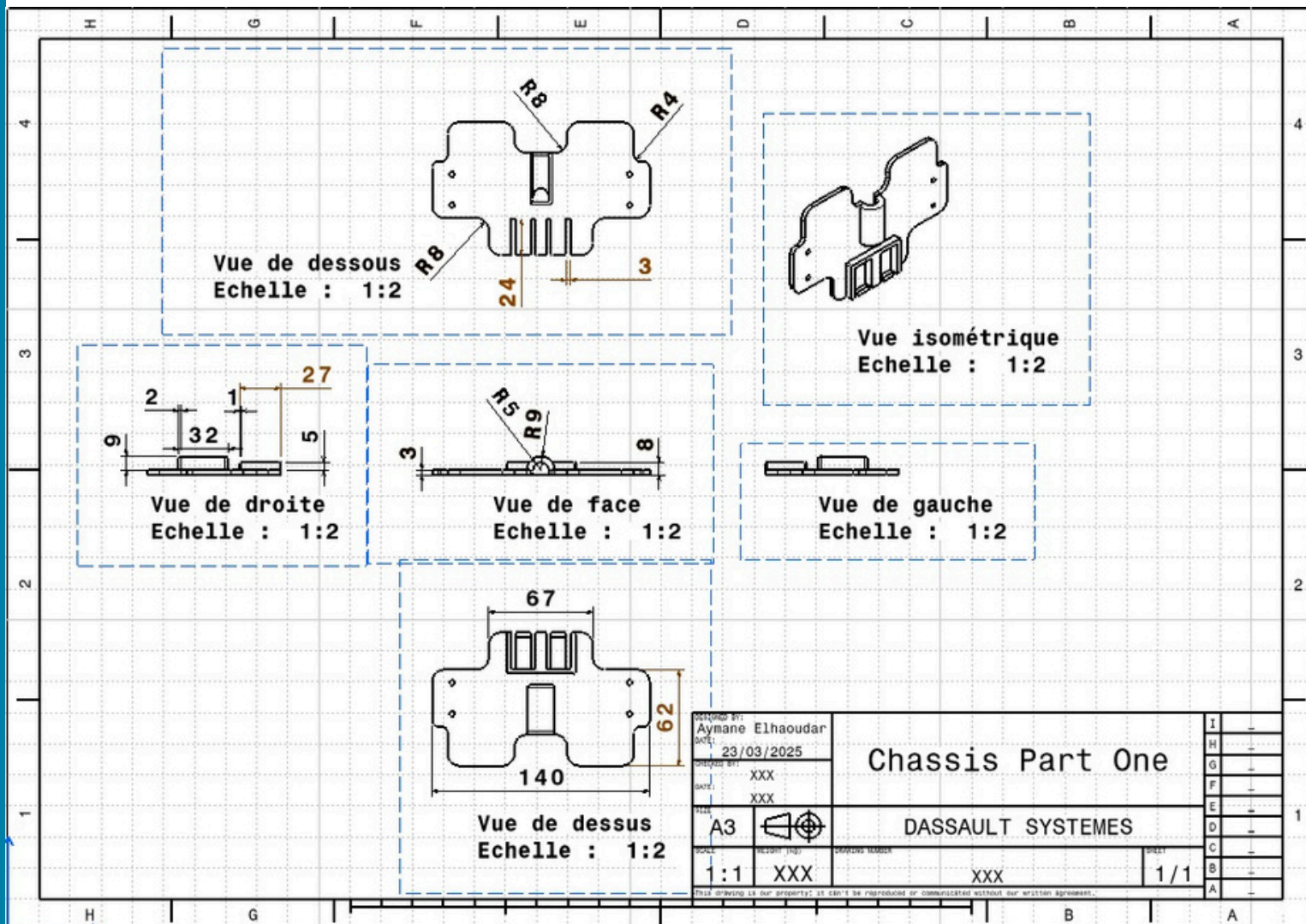
$$\text{On a: } \beta \neq \pi/2.$$

Donc:

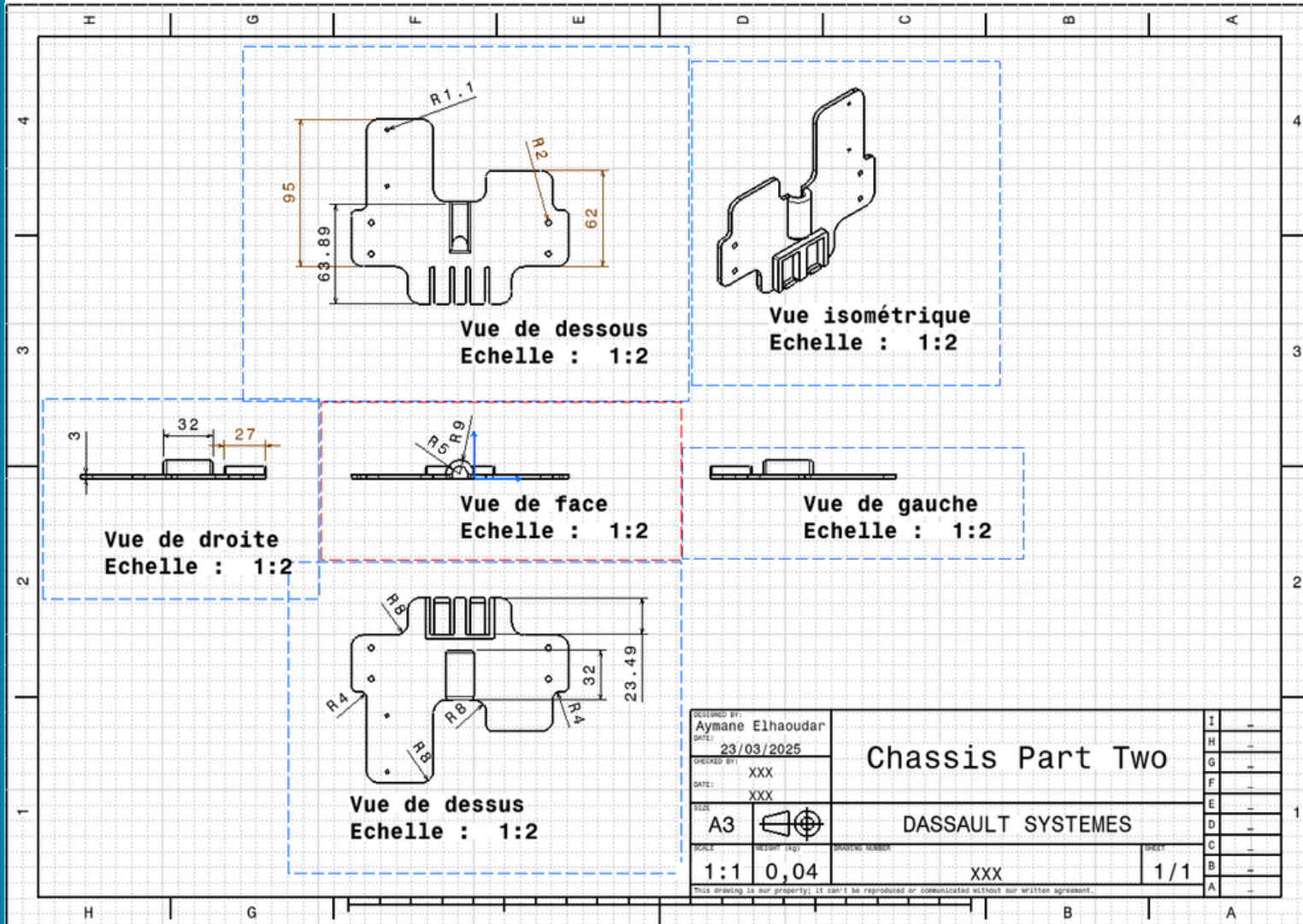
$$\alpha = \text{atan2}(a_y, a_x) \quad \text{d'où } \alpha = \theta_4$$

$$\beta = \text{atan2}(-a_z, \sqrt{a_x^2 + a_y^2}) \quad \text{d'où } \beta = \theta_5$$

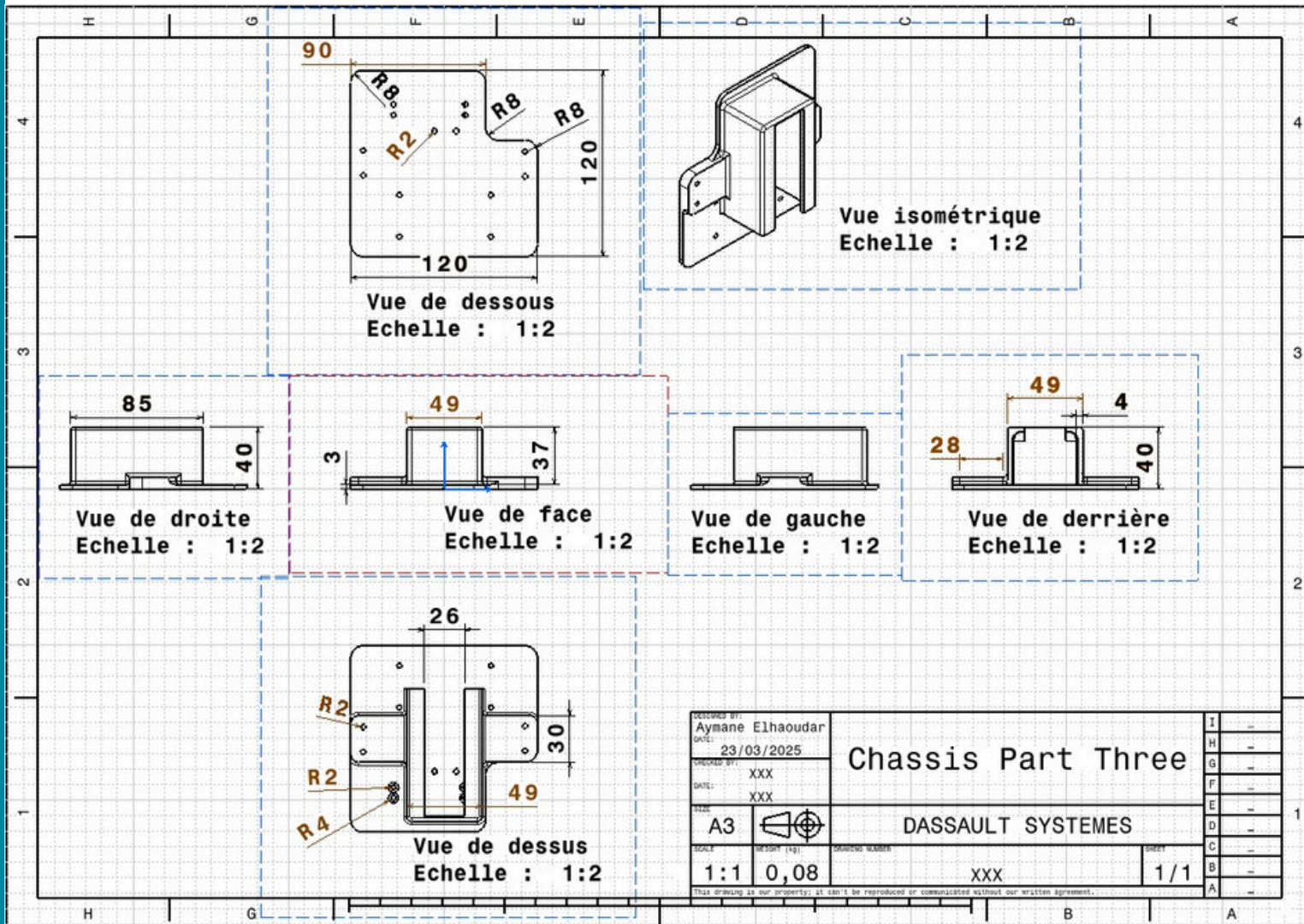
$$\gamma = \text{atan2}(b_z, c_z) \quad \text{d'où } \gamma = \pi/2$$



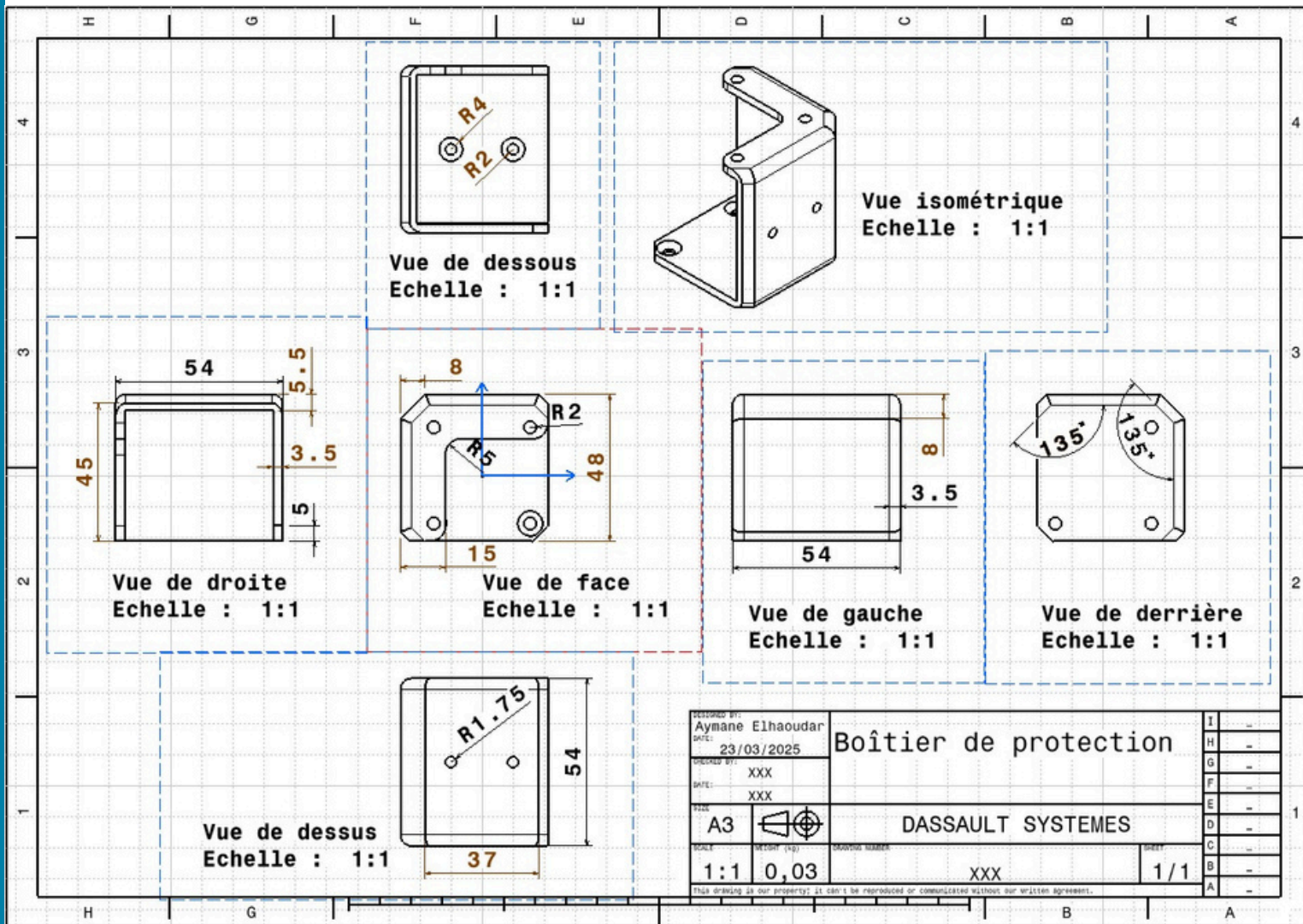
Plan de fabrication de la pièce du châssis un



Plan de fabrication de la pièce du châssis deux



Plan de fabrication de la pièce du châssis trois



Plan de fabrication du boîtier de protection du moteur

2.2 Contraintes du Robot

Le robot devra être capable de réaliser une partie de l'opération "ES sous caisse" pendant 1 heure, tout en garantissant une autonomie de 4 heures pour assurer des performances constantes. Il devra parcourir une distance de 200 mètres dans un espace restreint, avec les caractéristiques suivantes :

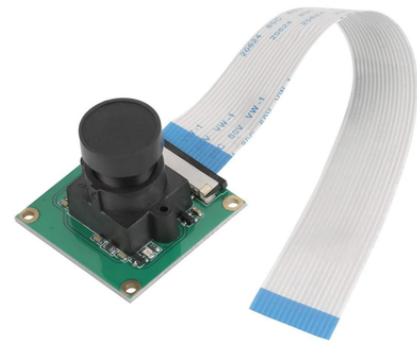
- *Largeur disponible : 1435 mm (écartement des rails).*
- *Hauteur sous caisse : 1500 mm.*

Le robot se déplacera à une vitesse de 1 m/s (3-5 km/h), avec une accélération pouvant atteindre 20 % de plus si nécessaire, afin de suivre le rythme des opérations humaines. Son rôle sera de détecter les pannes et de localiser leur position sous les rames TGV, facilitant ainsi l'intervention des techniciens.

3. Bibliographie :

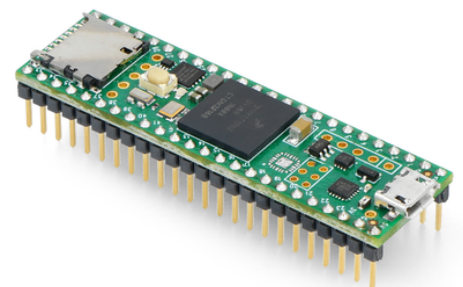
Système de Navigation : Caméra Raspberry Pi V3 :

- **Caméra Raspberry Pi V3:** Suivi d'un opérateur via l'algorithme OpenCV-KCF.
- **Détection d'Obstacles :** Modèle Coconames pour identifier les objets (plantes, personnes, obstacles).



Unité Esclave (Teensy 4.1) :

- **Contrôle moteur :** Gestion précise des moteurs et capteurs.
- **Communication :** Réception des commandes via UART pour une coordination optimale.



Stepper Motor :

- **Contrôle précis :** Ils permettent un contrôle précis des mouvements du bras.



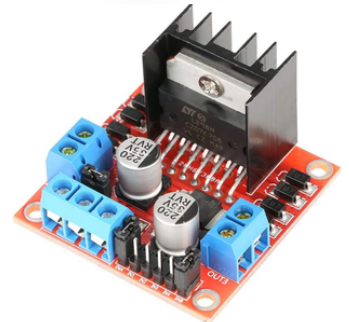
Capteurs LiDAR pour la navigation autonome et la cartographie 3D

- **Navigation autonome** : Cartographie 3D et détection d'obstacle
- **Modèle RPLIDAR A3M1** : Portée de 12m et balayage à 360°.



L298N Motor Driver Module

- **Contrôle précis** : Gestion de la vitesse et direction de deux moteurs DC.
- **Performance** : Supporte jusqu'à 35V et 2A par canal pour une fiabilité maximale.



Unité Maître (Raspberry Pi 5) :

- **Traitement avancé** : Analyse des données capteurs et caméra.
- **Intelligence embarquée** : Exécution d'algorithmes IA (détection d'objets, reconnaissance vocale).
- **Communication globale** : Connexion via MQTT/WiFi pour une intégration fluide.



IMU (Unité de Mesure Inertielle)

Une IMU est un capteur composé d'une combinaison d'accéléromètre, de gyroscope et de magnétomètre.

- Compenser les erreurs de positionnement et d'orientation.
- Améliorer la précision de la cartographie.



Module LoRa RFM9xW433/868/915MHz

- **Communication longue portée** : Transmission stable jusqu'à 250m.
- **Basse consommation** : Idéal pour les systèmes embarqués et autonomes.



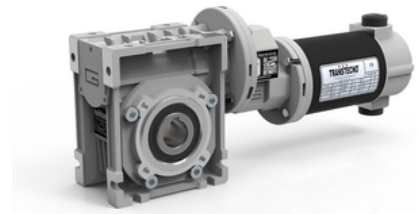
A4988 Stepper Motor Drivers

- **Le Control des moteurs pas a pas**



Motoreducteur

- **Fournission du couple** : Il fournit le couple necessaire au differents fonctions du robot d inspection sous caisse en fosse



Jetson Nano

- Gere le control global du robot et le traitement des donnees de la camera, ses capacites de traitement plus puissantes, essentiels pour gerer les calculs



5. Choix de Micro-controleur

L'ESP32 est un microcontrôleur puissant et polyvalent, adapté aux applications de robotique. Ce document détaille les critères de sélection et de dimensionnement de l'ESP32 pour un robot d'inspection sous caisse.

Exigences du Robot

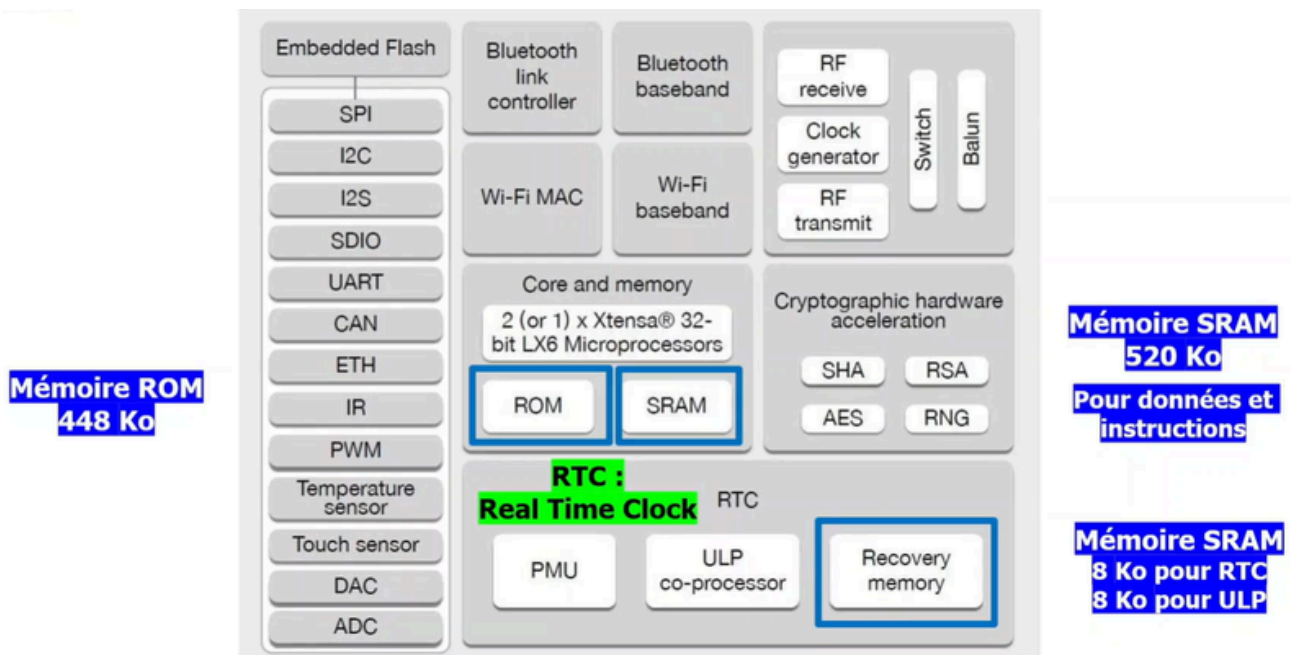
Le robot doit :

- Se déplacer sous les véhicules pour effectuer une inspection visuelle et transmettre les données en temps réel.
- Intégrer des capteurs (caméra, ultrason, IMU) pour l'autonomie et la navigation.
- Communiquer sans fil avec une interface utilisateur.

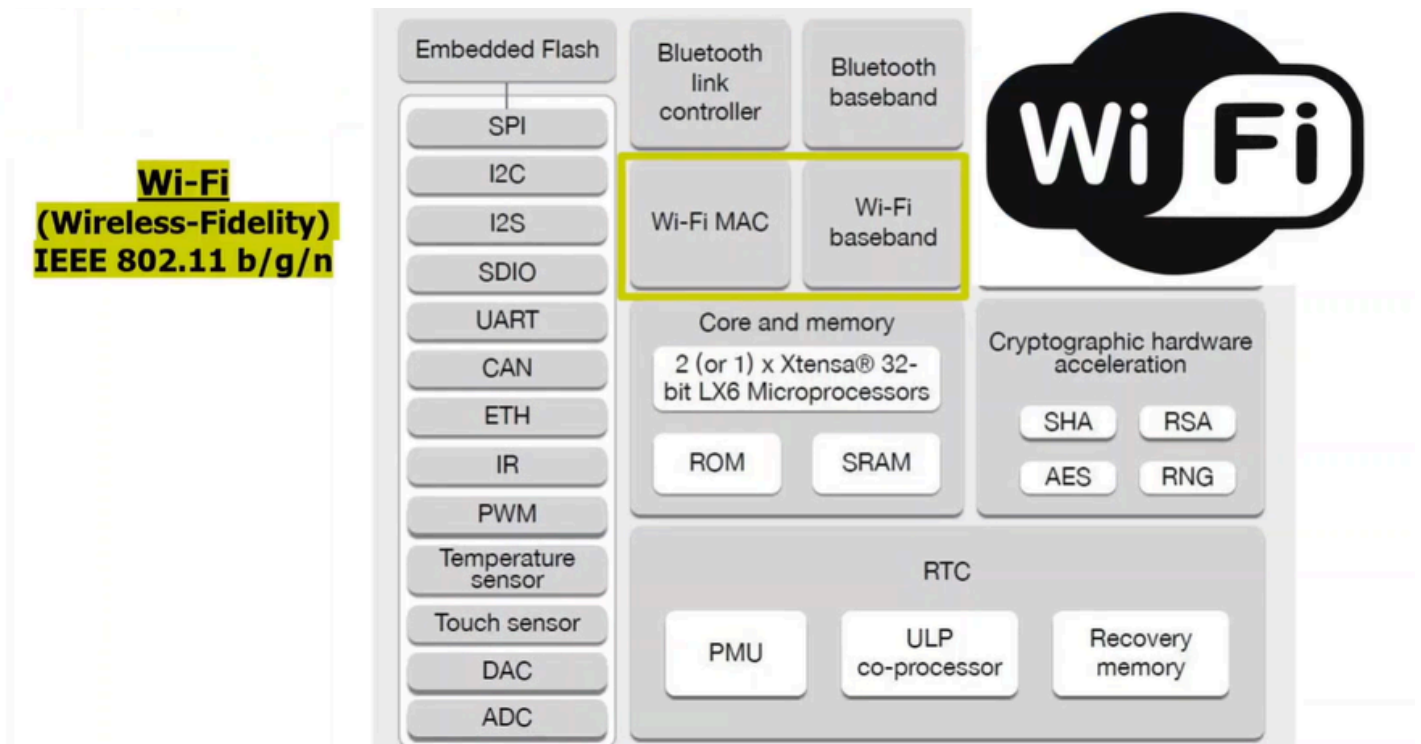
L'ESP32 est choisi pour :

- **Sa puissance de calcul** : Dual-core Xtensa LX6 (240 MHz).
- **Sa connectivité** : Wi-Fi et Bluetooth pour la transmission de données.
- **Ses GPIO polyvalents** : Permet la connexion de plusieurs capteurs et actionneurs.
- **Sa faible consommation** : Adapté aux robots alimentés sur batterie.

5.1 Les Mémoires



5.2 Les liaisons radio : Wi-Fi / Bluetooth



5.3 Alimentation et Consommation

L'ESP32 fonctionne entre 3,3V et 5V.

- Consommation en veille : ~10 mA
- Consommation en fonctionnement : ~160-260 mA
- Une batterie LiPo 7,4V 2000mAh avec un régulateur de tension 3,3V est recommandée.

6. Dimensionnement du Unité Maître (Raspberry Pi 5) :

L'ESP32 est un microcontrôleur puissant et polyvalent, adapté aux applications de robotique. Ce document détaille les critères de sélection et de dimensionnement de l'ESP32 pour un robot d'inspection sous caisse.

6.1 Choix du Raspberry Pi 5

Le Raspberry Pi 5 agit comme unité maître pour :

- Coordonner les communications entre les capteurs et l'interface utilisateur.
 - Traiter les images capturées par la caméra du robot grâce à sa puissante GPU et CPU.
 - Gérer les algorithmes d'intelligence artificielle pour l'analyse des images et la prise de décision autonome.
 - Assurer le stockage des données en local et leur transmission vers un serveur distant.
5. Alimentation et Consommation

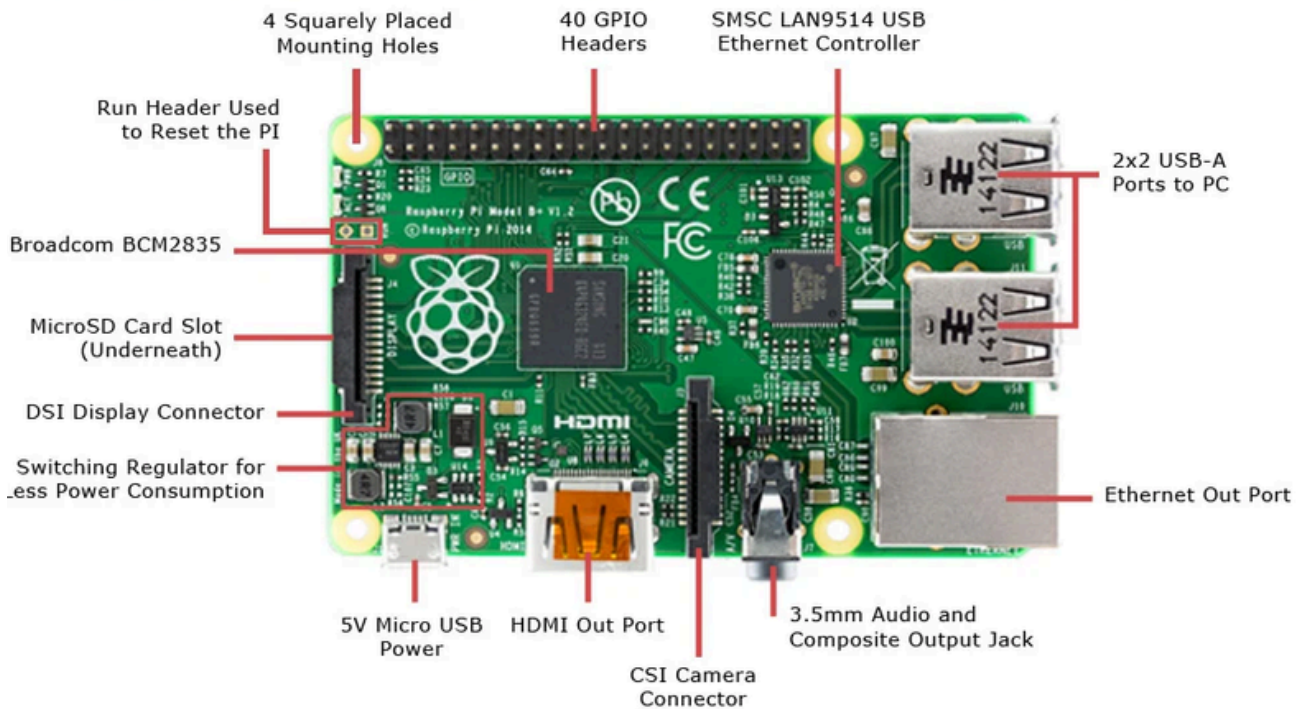
Table 1: Key Timing Parameters

Speed Grade	Clock Rate (MHz)	Data Rate (Mb/s/pin)	WRITE Latency		READ Latency	
			Set A	Set B	DBI Disabled	DBI Enabled
-046	2133	4266	18	34	36	40

6.2 Raspberry Pi 5

- Fonctionne sous 5V via USB-C.
- Consommation moyenne : 5W - 8W en fonction de la charge.
- Une batterie LiPo 12V avec convertisseur 5V/3A est recommandée pour une autonomie suffisante.

GPIO Pinout Diagram



8. Dimensionnement du Caméra :

• Paramètres clés à considérer :

Résolution : Doit être assez élevée pour repérer les détails des pièces mécaniques.

Champ de vision (FOV - Field of View) : Dépend de la hauteur à laquelle la caméra est montée et de la largeur de la zone à inspecter.

Fréquence d'image (FPS - Frames Per Second) : Doit être suffisante pour capturer une inspection fluide lorsque le robot se déplace.

Sensibilité à la lumière : Utile si l'inspection se fait en environnement sombre.

Features/Specs	Camera Module 3
Release Date	January 2023
Image Sensor	IMX708
Image Resolution	4608 × 2592
Video Resolution	1080p @ 50fps
Fixed/Changeable Lens	Fixed

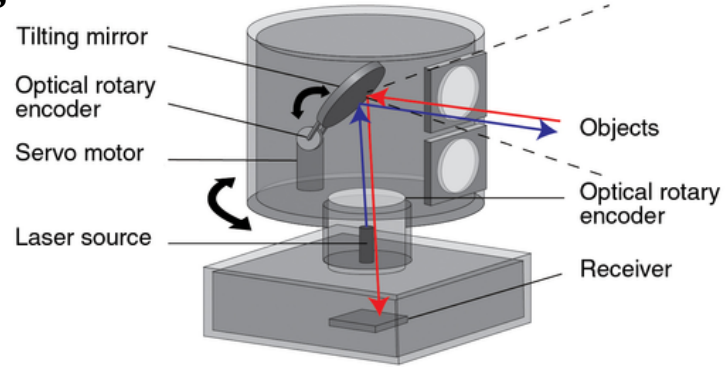
AutoFocus	Yes (PDAF)
View Angle	75° or 120° (Wide)
Variants	4
Library/Driver	Picamera2 (Python)
Interface	15-way CSI
Compatibility	All Raspberry Pi Models

9. Fonctionnement du LiDAR :

Le **LiDAR** (Light Detection and Ranging) fonctionne en envoyant **des impulsions laser** et en mesurant le temps que met la lumière à revenir après réflexion sur une surface. Il permet ainsi de créer **une cartographie précise** de l'environnement sous la caisse du train ou du véhicule.

A. Navigation et Évitement d'Obstacles

- Le robot utilise le LiDAR pour cartographier l'espace sous la caisse en temps réel.
- Il détecte la présence d'obstacles (câbles, supports, débris) et adapte sa trajectoire.
- Un LiDAR 360° permet une vision complète pour éviter les collisions.



B. Cartographie 3D et Reconstruction du Châssis

- En balayant la zone sous la caisse, le LiDAR génère un nuage de points 3D représentant fidèlement les structures sous le train.
- Cette cartographie est comparée à un modèle de référence (CAO ou données antérieures) pour détecter des anomalies.

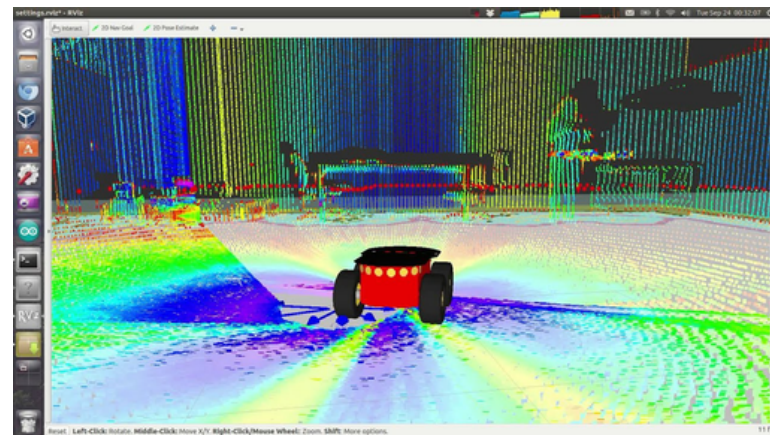


Figure 1. Perception 3D d'un robot mobile à l'aide du LiDAR

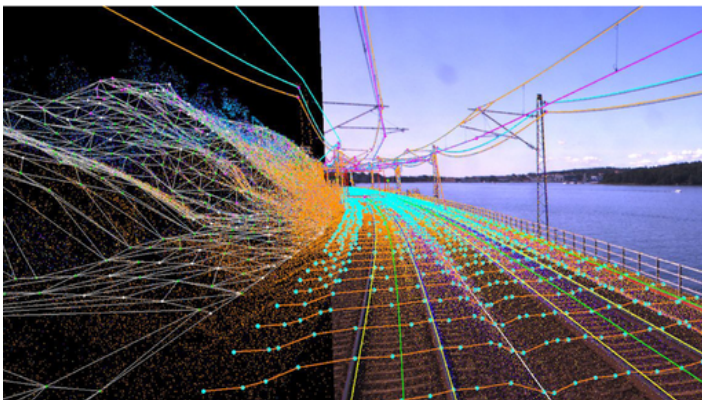
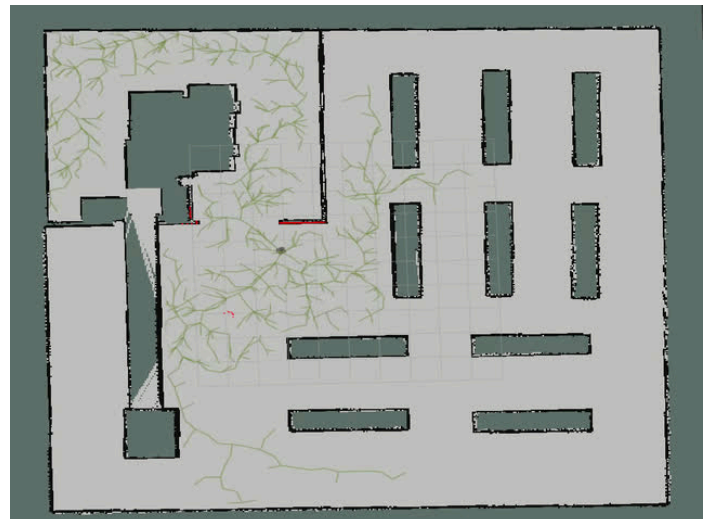
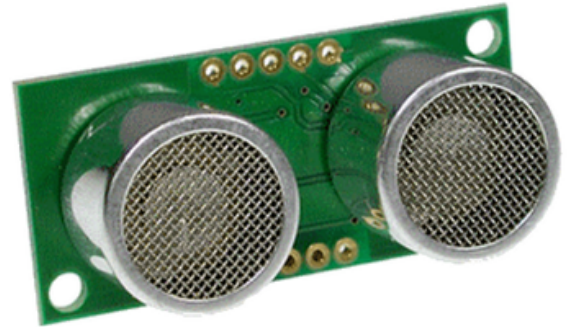


Figure 2. Inspection Ferroviaire Assistées par LiDAR



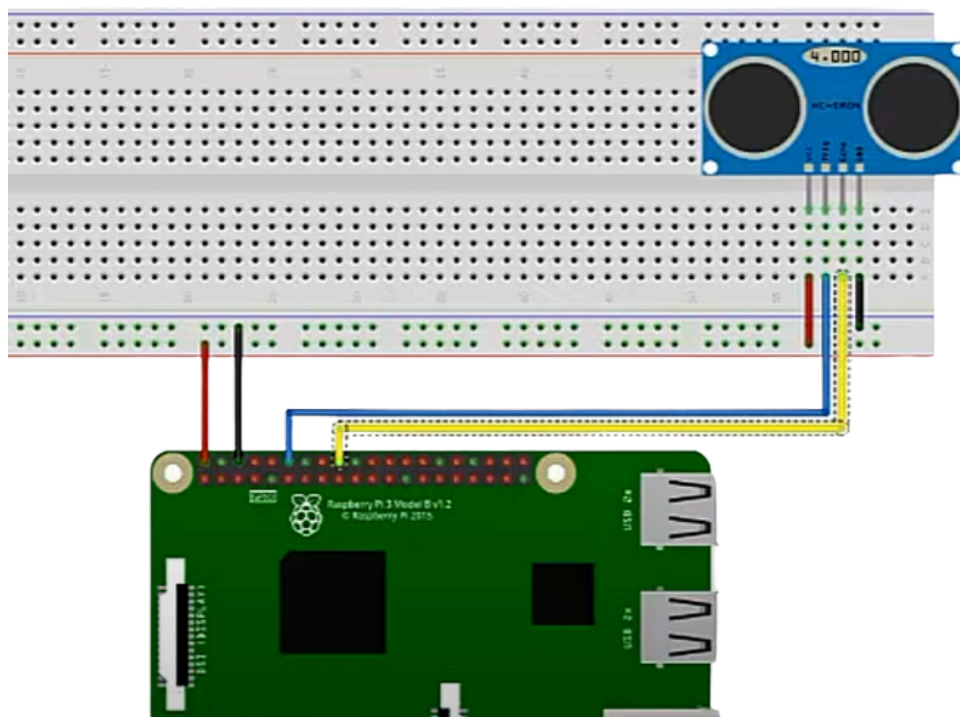
Ultrasons:SRF10

Le capteurs ultrason seront répartis sur le bras afin de détecter les obstacles qui risque de toucher le bras au cours de son fonctionnement et donc s'arrêtera si ils sont présent



Caractéristiques

- *Gain analogique maximum : Variable de 40 à 700 en 16 pas*
- *Connexion : Bus I2C standard*
- *Temps : Écho entièrement chronométré, libérant le contrôleur hôte de la tâche*
- *Unités : Plage indiquée en μ S, cm ou pouces*



```
1 #include <Wire.h>
2 #define SRF10_ADDRESS 0x70 // Adresse par défaut du capteur SRF10
3 void setup() {
4   Wire.begin(); // Initialisation de la communication I2C
5   Serial.begin(9600); // Initialisation de la communication série
6 }
7 void loop() {
8   // Envoyer une commande pour démarrer une mesure de distance
9   Wire.beginTransmission(SRF10_ADDRESS);
10  Wire.write(0x00); // Commande pour mesurer la distance
11  Wire.endTransmission();
12  // Attendre la fin de la mesure (100 ms suffisent généralement)
13  delay(100);
14  // Lire la distance mesurée depuis le capteur (en centimètres)
15  Wire.requestFrom(SRF10_ADDRESS, 2); // Le capteur renvoie 2 octets
16  int highByte = Wire.read();
17  int lowByte = Wire.read();
18  int distance = (highByte << 8) + lowByte;
19  // Afficher la distance mesurée
20  Serial.print("Distance: ");
21  Serial.print(distance);
22  Serial.println(" cm");
23  // Attendre un court délai avant de refaire une mesure
24  delay(500);
25 }
26
```

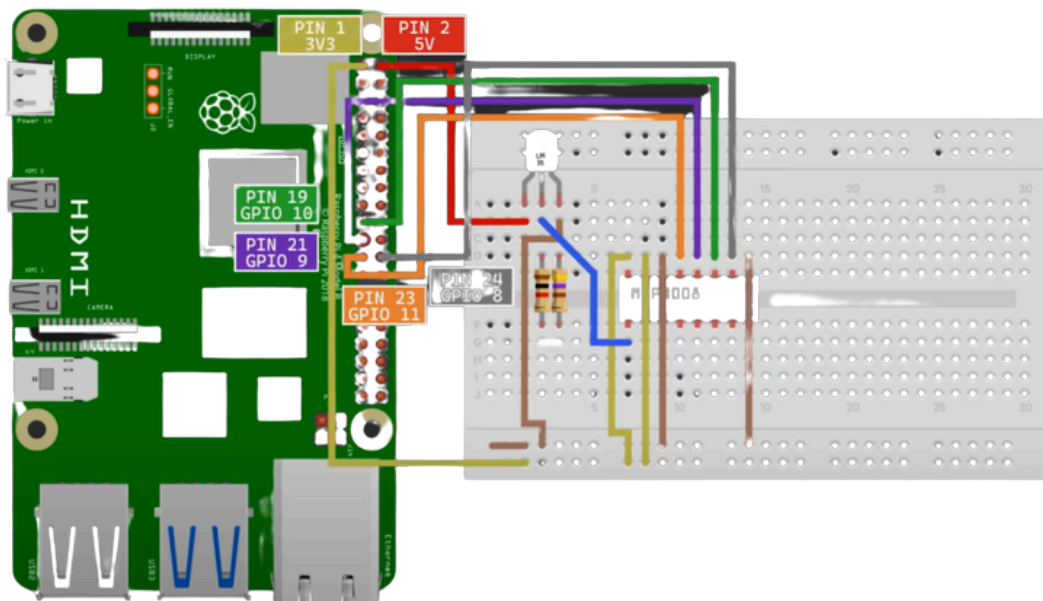
Capteur de température LM35:

Les capteurs de température seront disposés avec les moteurs afin de mesurer leurs températures tout au long du fonctionnement du robot, ainsi lorsque la température sera supérieur à une température prédéfini le système de refroidissement démarrera automatiquement (ventilateur) et le robot pourra terminer sa tâche en toute sécurité



Fiche technique

Alimentation nominale	4V à 30V
Liaison de donnée	Analogique
Précision	$\pm 0.75^{\circ}\text{C}$
Etendue de mesure	0°C à 100°C



```
1 const int lm35Pin = A0;
2 // Seuil de température à ne pas dépasser
3 const float seuilTemperature = 50.0; // 50 degrés Celsius, à ajuster selon vos
4 besoins
5 // Broche pour contrôler la LED
6 const int ledPin = 13;
7 void setup() {
8 // Initialisation de la communication série
9 Serial.begin(9600);
10 // Configuration de la broche de la LED en sortie
11 pinMode(ledPin, OUTPUT);
12 }
13 void loop() {
14 // Lecture de la tension sur la broche analogique à laquelle le LM35 est
15 connecté
16 int sensorValue = analogRead(lm35Pin);
17 // Conversion de la valeur lue en tension en température en degrés Celsius
18 f
19 loat temperature = (sensorValue * 5.0) / 1024.0 * 100.0;
20 // Affichage de la température mesurée
21 Serial.print("Temperature: ");
22 Serial.print(temperature);
23 Serial.println(" degrees Celsius");
24 // Si la température dépasse le seuil spécifié
25 if (temperature > seuilTemperature) {
26 // Allumer la LED pour indiquer une température élevée
27 digitalWrite(ledPin, HIGH);
28 } else {
29 // Éteindre la LED si la température est inférieure ou égale au seuil
30 digitalWrite(ledPin, LOW);
31 }
32 delay(1000); }
33
```

Capteur infrarouge Sharp GP2Y0A02YK0F avec buzzer:

Le capteur infrarouge- buzzer sera mis au devant du robot afin de détecter si quelqu'un est présent sur son chemin, si c'est le cas le buzzer fonctionne pour alerter la personne présente.

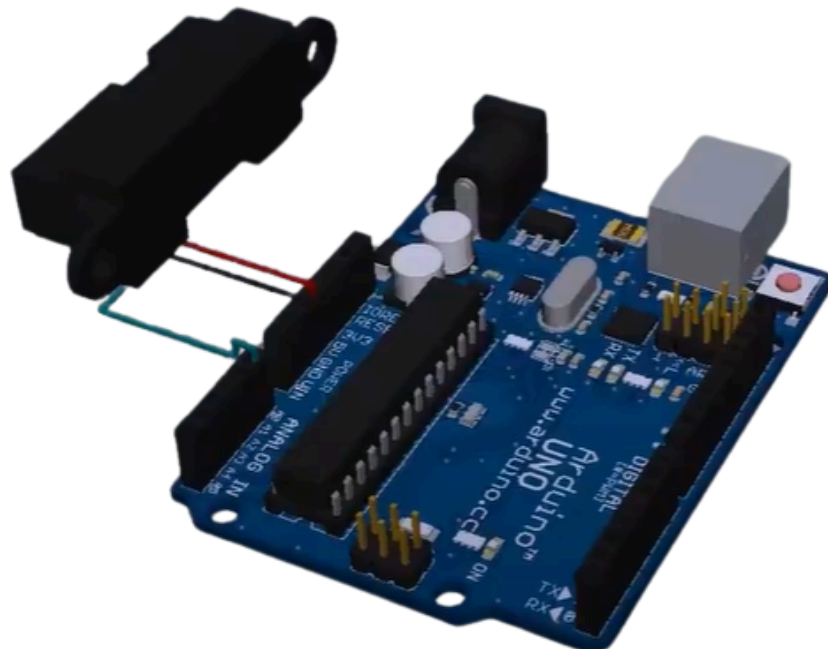


Fonctionnalité

Le GP2Y0A02YK0F est un capteur de distance basé sur la triangulation. Il comprend un PSD, une IRED et un circuit de traitement du signal. Sa précision n'est pas affectée par le matériau, la température ou le temps de mesure.

Paramètres techniques :

- ***Plage de mesure de distance :*** 20 à 150 cm
- ***Type de sortie de signal :*** signal analogique de tension
- ***Taille d'emballage :*** 29,5 × 13 × 21,6 mm
- ***Consommation électrique :*** 33 mA nominal
- ***Tension d'alimentation :*** 4,5 à 5,5 V



```
1 // Broche à laquelle le capteur infrarouge est connecté
2 const int irSensorPin = 9;
3 // Broche à laquelle le buzzer est connecté
4 const int buzzerPin = 2;
5 void setup() {
6 // Initialisation de la communication série
7 Serial.begin(9600);
8 // Configuration de la broche du capteur infrarouge en entrée
9 pinMode(irSensorPin, INPUT);
10 // Configuration de la broche du buzzer en sortie
11 pinMode(buzzerPin, OUTPUT);
12 }
13 void loop() {
14 // Lecture de l'état du capteur infrarouge
15 int irSensorValue = digitalRead(irSensorPin);
16 // Si un obstacle est détecté (signal infrarouge détecté)
17 if (irSensorValue == HIGH) {
18 // Activer le buzzer
19 digitalWrite(buzzerPin, HIGH);
20 // Afficher un message sur le moniteur série
21 Serial.println("Obstacle détecté !");
22 } else {
23 // Désactiver le buzzer
24 digitalWrite(buzzerPin, LOW);
25 }
26 // Attente avant la prochaine lecture
27 delay(100); // Ajustez selon vos besoins
28 }
29
```

LoRa Transceivers

RFM9xW433/868/915MHz, ISM LoRa Transceivers, HopeRF

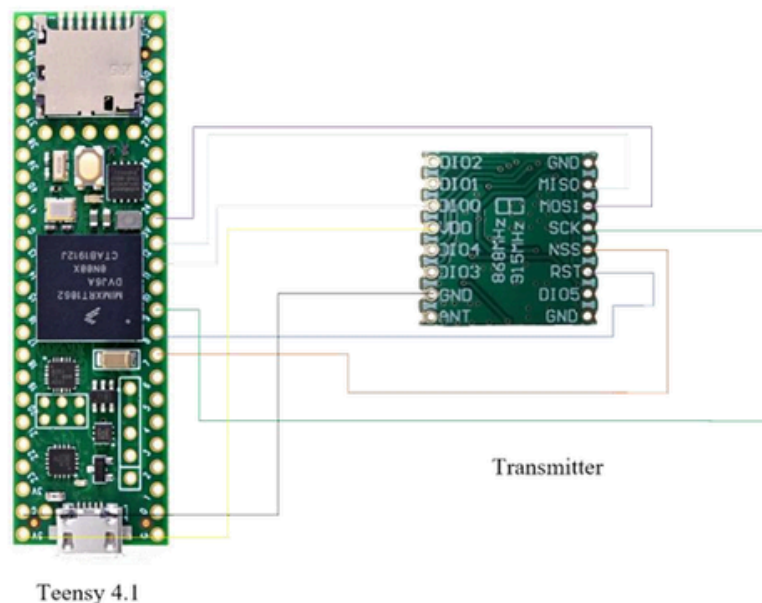
Les modules émetteurs-récepteurs RFM9xW sont dotés de la puce émetteur-récepteur Semtech SX1272 qui utilise une variante du codage Chirp Spread Spectrum appelée LoRa™, pour améliorer la sensibilité du récepteur et ainsi atteindre une portée étendue. Le module peut également utiliser les schémas de modulation OOK et FSK pour maintenir la compatibilité avec d'autres normes telles que IEEE 802.15.4 et wMBUS. LoRa présente une immunité aux interférences accrue par rapport au FSK conventionnel, offrant une capacité de coexistence améliorée dans les canaux encombrés.



Spécifications :

Attribut	Valeur
Puissance de sortie maximale	+13dBm
Sensibilité du récepteur	-148dBm
Type de protocole	Lorraine
Interfaces de bus prises en charge	IPS
Tension d'alimentation	3,3 V
Dimensions	16x16x1.8mm

Connections :



Conception et Câblage de notre Contrôle Robotisé à Base de Raspberry Pi et ESP32

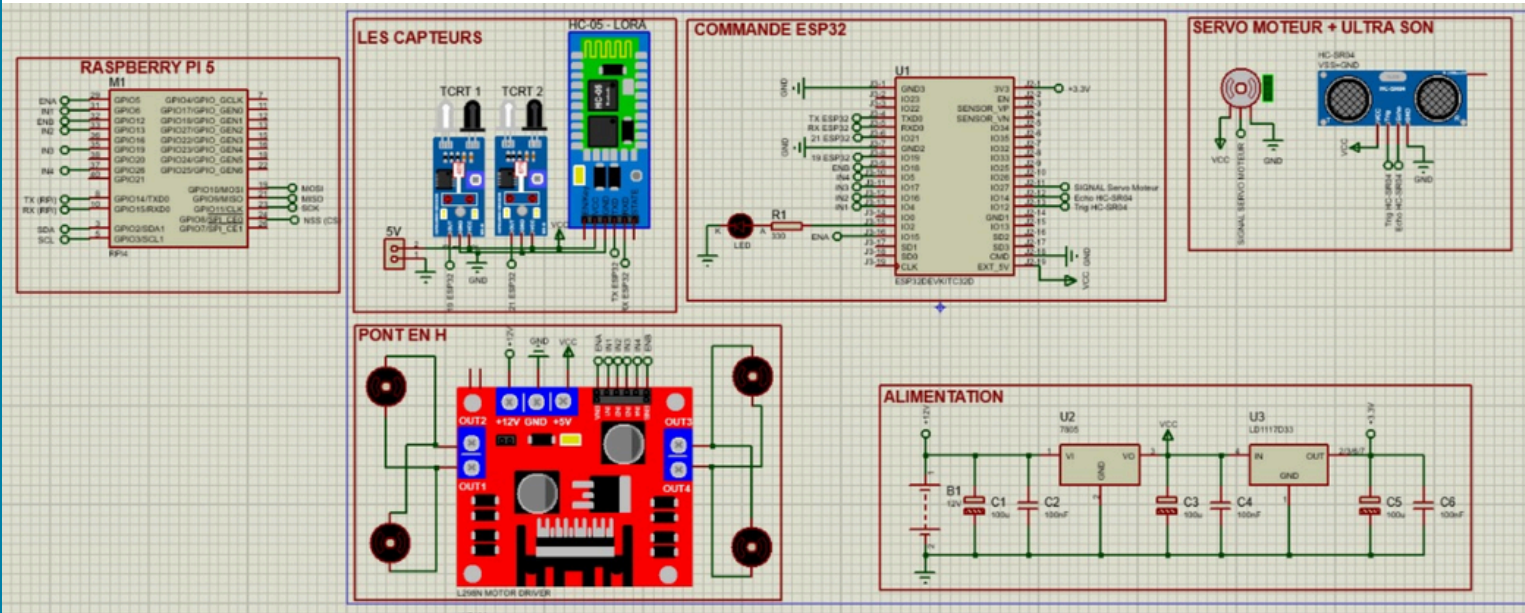
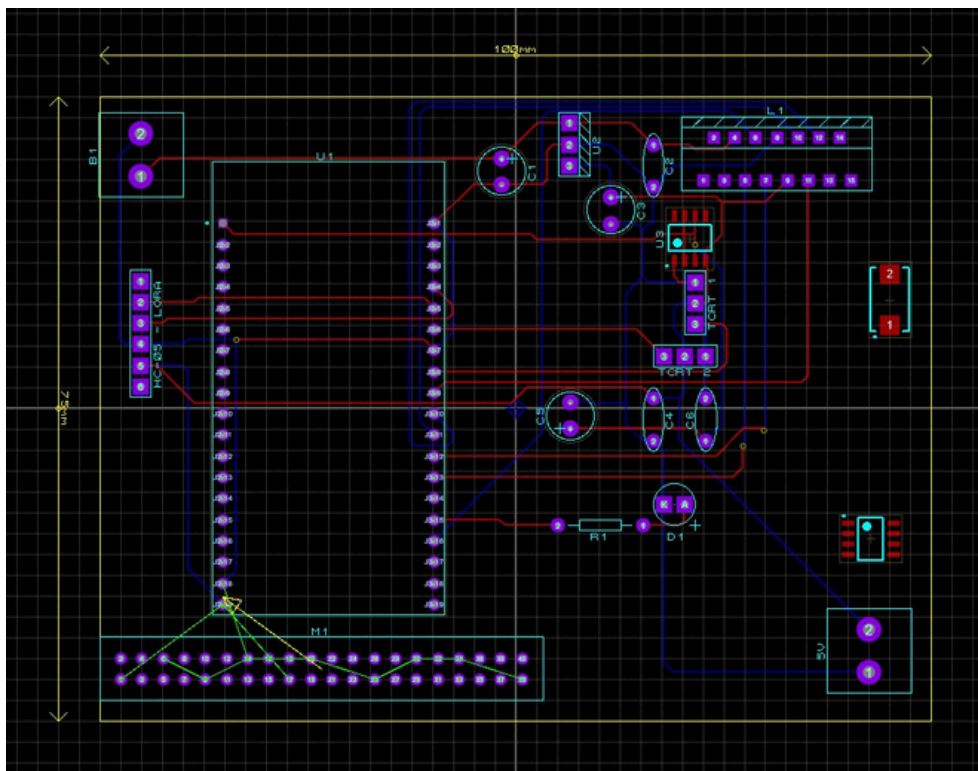
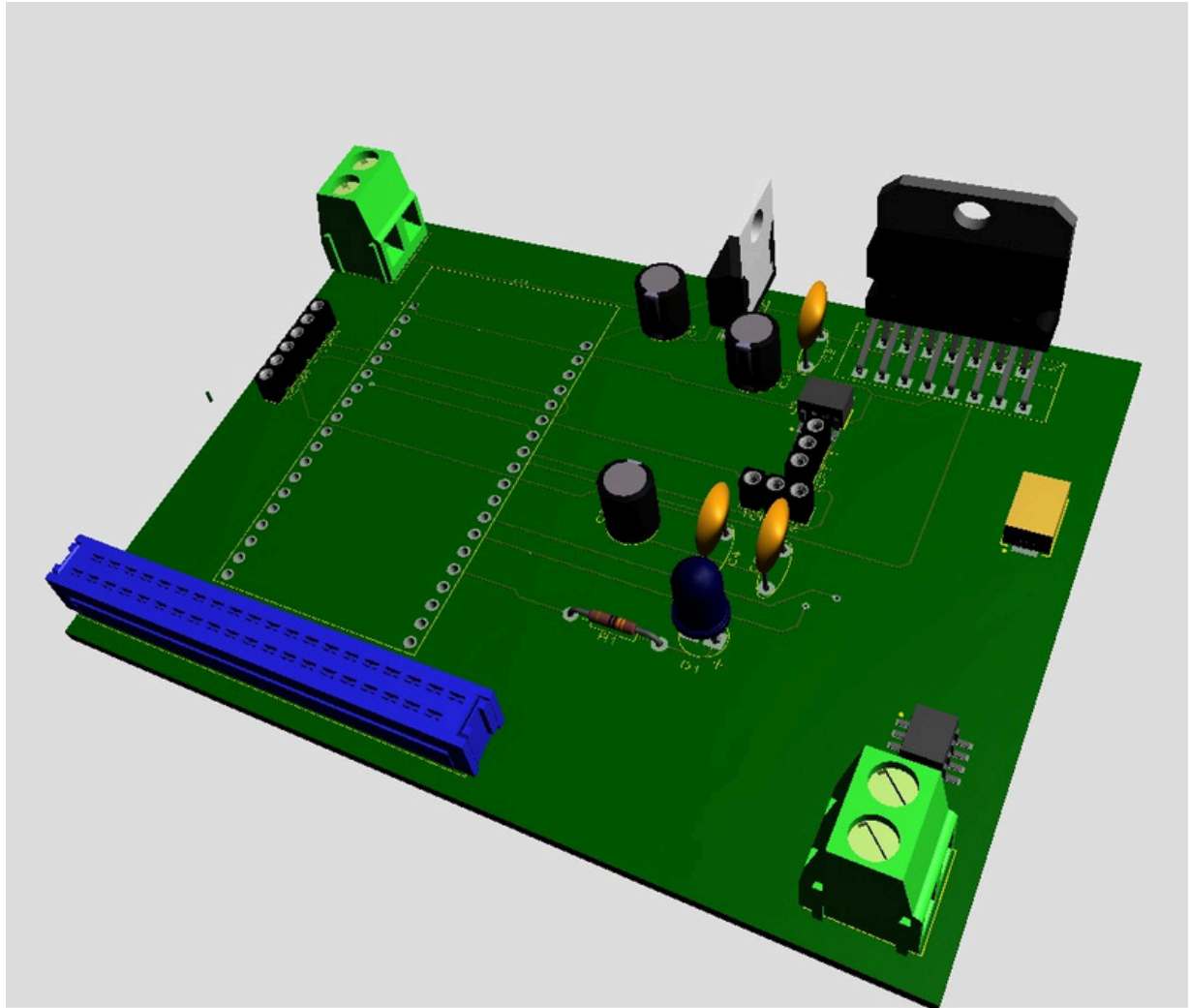


Schéma du Circuit du Système de Contrôle Robotisé



PCB Layout pour le Système de Contrôle Robotisé

Conception et Câblage de notre Contrôle Robotisé à Base de Raspberry Pi et ESP32



Visualisation 3D du Circuit Imprimé de Contrôle Robotisé

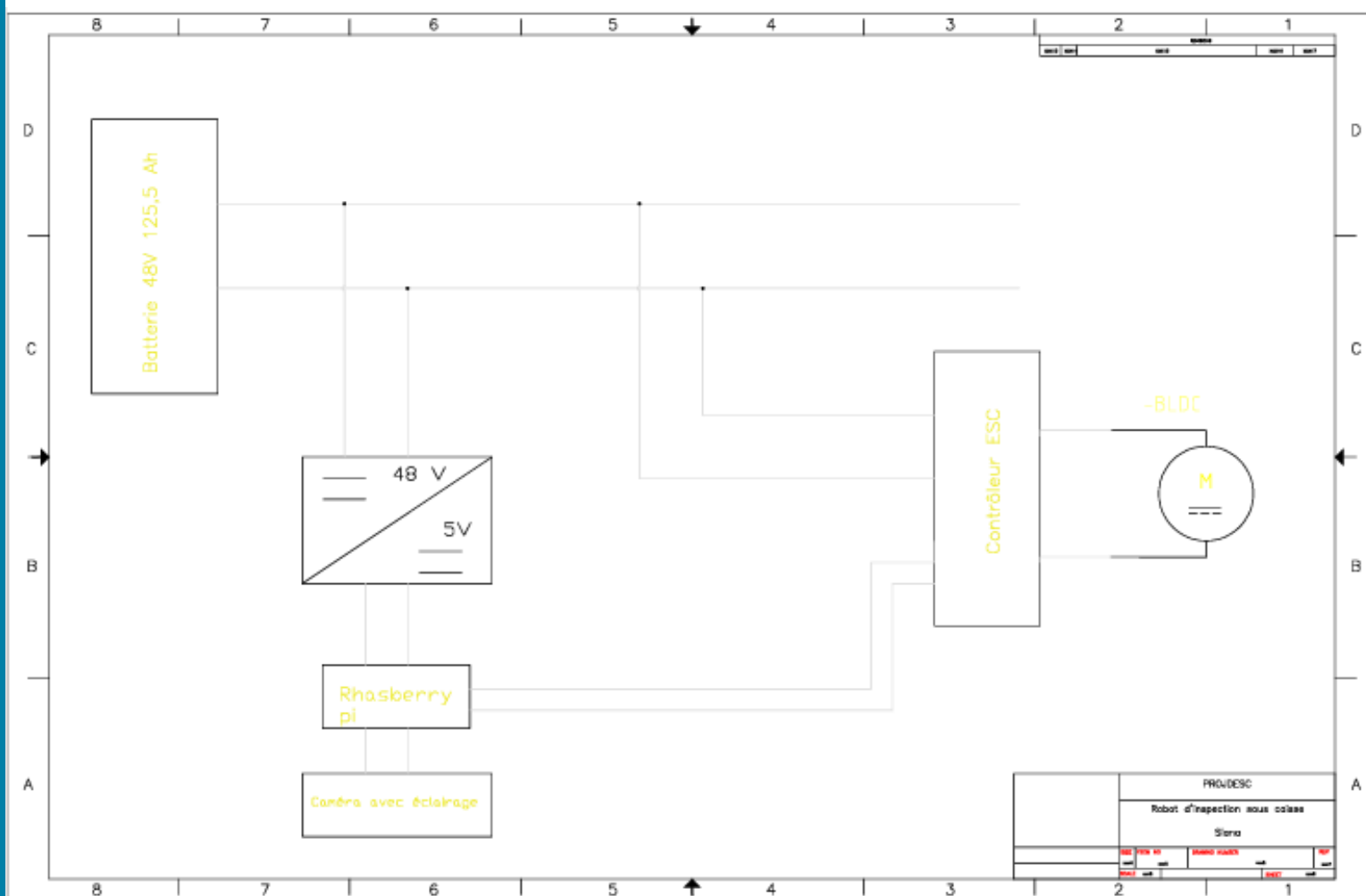


Schéma de câblage du système de gestion d'énergie pour moteur et contrôleur ESC

Caméra et imagerie :

1. Commande:

- Capture d'image :

```

1  from picamera import PiCamera
2  import time
3
4  camera = PiCamera()
5  time.sleep(2)
6  camera.resolution = (1280, 720)
7  camera.vflip = True
8  camera.contrast = 10
9
10 file_name = "/home/pi/Pictures/img_" + str(time.time()) + ".jpg"
11 camera.capture(file_name)
12

```

Ce code consiste à capturer des images à l'aide d'une caméra raspberry et les stocker dans un fichier choisis

- traitement d'imagerie :

```

1  import cv2
2  import numpy as np
3
4  # Load reference image
5  ref_img = cv2.imread('ref_img.jpg')
6  ref_gray = cv2.cvtColor(ref_img, cv2.COLOR_BGR2GRAY)
7
8  # Capture image using Raspberry Pi camera
9  cap_img = cv2.imread('img_*.jpg')
10 cap_gray = cv2.cvtColor(cap_img, cv2.COLOR_BGR2GRAY)
11
12 # Subtract the reference image from the captured image
13 diff_img = cv2.absdiff(ref_gray, cap_gray)
14
15 # Apply thresholding to the difference image
16 threshold = 50
17 _, diff_img = cv2.threshold(diff_img, threshold, 255, cv2.THRESH_BINARY)
18
19 # Find contours in the difference image
20 contours, _ = cv2.findContours(diff_img, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
21
22 # Draw rectangles around the detected differences
23 for contour in contours:
24     x, y, w, h = cv2.boundingRect(contour)
25     cv2.rectangle(cap_img, (x, y), (x + w, y + h), (0, 0, 255), 2)
26
27 # Display the captured image with the detected differences
28 cv2.imshow('Captured Image', cap_img)
29 cv2.waitKey(0)
30 cv2.destroyAllWindows()
31

```

Ce code consister à comparer deux images, une capturer par la caméra raspberry et l'autre stocker en mémoire et détecte les différences entre les images; afin de détecter les différences.

- transmission d'image :

```

1  import socket
2  import time
3
4  # Initialize LoRa socket
5  s = socket.socket(socket.AF_LORA, socket.SOCK_RAW)
6  s.setsockopt(socket.SOL_LORA, socket.SO_DRATE_REGION, 1)
7  s.setsockopt(socket.SOL_LORA, socket.SO_LORA_BANDWIDTH, 0) # 125 kHz
8  s.setsockopt(socket.SOL_LORA, socket.SO_LORA_SPREADING_FACTOR, 7) # 6-12
9  s.setsockopt(socket.SOL_LORA, socket.SO_LORA_CODING_RATE, 4) # 4/5
10 s.setsockopt(socket.SOL_LORA, socket.SO_LORA_PREAMBLE, 8) # Default
11 s.setsockopt(socket.SOL_LORA, socket.SO_LORA_IQ, 0) # Default
12
13 # Open a LoRa socket
14 s.bind(1)
15
16 # Transmit captured image
17 with open('img_*.jpg', 'rb') as f:
18     data = f.read()
19     s.send(data)
20     print("Image sent!")
21
22 # Close the socket
23 s.close()
24
  
```

On utilisera le module LORA pour transmettre les images de la carte de traitement vers un moniteur éloigner.

- Détection de type d'objet :

```

1  import cv2
2
3  # Load the Haar cascades classifier
4  face_cascade = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
5
6  # Capture image using Raspberry Pi camera
7  cap_img = cv2.imread('img_x.jpg')
8
9  # Convert the image to grayscale
10 gray_img = cv2.cvtColor(cap_img, cv2.COLOR_BGR2GRAY)
11
12 # Detect faces in the image
13 faces = face_cascade.detectMultiScale(gray_img, scaleFactor=1.1, minNeighbors=5,
14
15 # Draw rectangles around the detected faces
16 for (x, y, w, h) in faces:
17     cv2.rectangle(cap_img, (x, y), (x+w, y+h), (0, 0, 255), 2)
18
  
```

```
18  
19 # Display the captured image with the detected faces  
20 cv2.imshow('Captured Image', cap_img)  
21 cv2.waitKey(0)  
22 cv2.destroyAllWindows()  
23
```

C'est une IA qui traite les images ou il y'a une difference afin de determiner le type de panne et envoie un rapport.

- ***Traitement des anomalies via Machine Learning dans le Cloud***

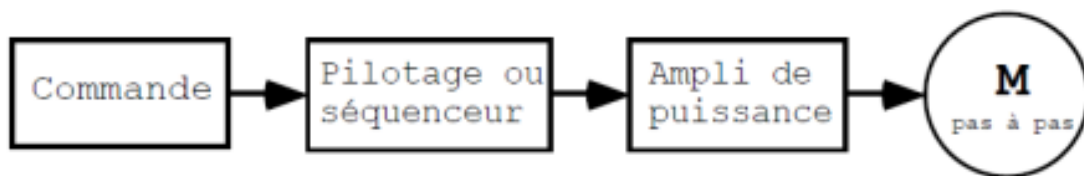
```
1 import tensorflow as tf  
2 import numpy as np  
3 import requests  
4  
5 # Charger un modèle d'IA pour la détection des anomalies (par exemple un modèle d  
6 model = tf.keras.models.load_model('anomaly_detection_model.h5')  
7  
8 # Charger une image depuis le cloud (exemple)  
9 image_data = requests.get("https://Train-cloud.com/Anomalies_data.jpg")  
10 image = np.array(image_data.content)  
11  
12 # Prétraiter l'image (dimensionner, normaliser, etc.)  
13 image = tf.image.resize(image, (224, 224))  
14 image = image / 255.0  
15  
16 # Prédiction de l'anomalie  
17 predictions = model.predict(np.expand_dims(image, axis=0))  
18  
19 # Si la probabilité d'anomalie dépasse un seuil, alerter la maintenance  
20 if predictions[0] > 0.5:  
21     print("Anomalie détectée sur la bougie du train. Alerte envoyée.")  
22
```

10. Commande du Robot d'Inspection Sous Caisse pour TGV

• Objectif du Système de Commande

Le système de commande contrôle le robot en mode autonome via des capteurs et intelligence embarquée (LIDAR), ou en mode télécommande, où le récepteur décode les signaux envoyés pour activer les actionneurs et exécuter les mouvements souhaités.

Mode autonome : Le robot navigue en se basant sur les capteurs et prend des décisions en fonction des obstacles détectés.

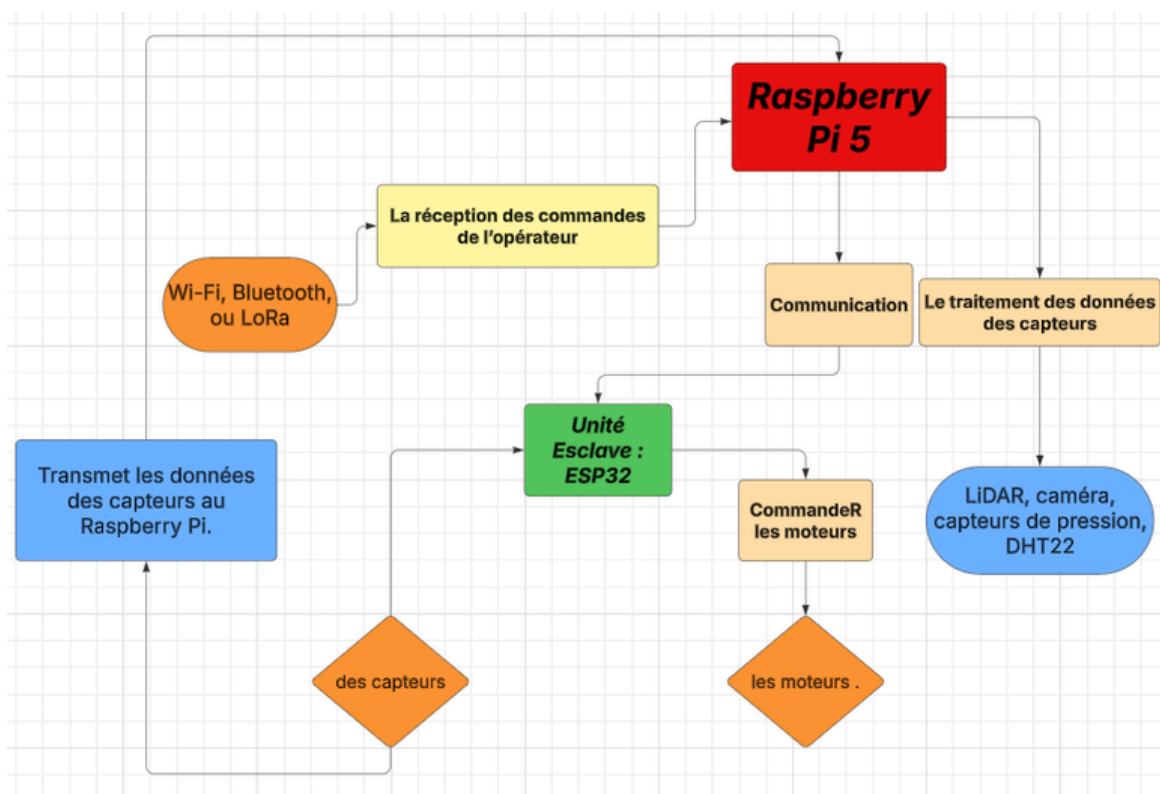


11. Architecture Globale du Système de Commande

11.1 Unité Maître : Raspberry Pi 5

Le Raspberry Pi 5 est le cœur du système de commande. Il gère :

- La réception des commandes de l'opérateur (via Wi-Fi ou bien LoRa).
- Le traitement des données des capteurs (LiDAR, caméra, capteurs de pression....).
- La communication avec l'unité esclave ESP32, qui contrôle les moteurs et autres actionneurs.



11.2 Unité Esclave : ESP32

L'ESP32 est chargé de l'exécution des commandes moteurs et de la gestion des capteurs.

- Il reçoit les ordres du Raspberry Pi 5 via Wi-Fi ou LoRa.
- Il commande les moteurs (direction et vitesse).
- Il collecte les données des capteurs et les envoie au Raspberry Pi.

Émetteur(télécommande)

- L'émetteur est l'appareil tenu par l'opérateur pour envoyer des signaux de commande au robot. Il est équipé de joysticks, de boutons de



Channels	6-10 (Default 6)
Model Type	Fixed-Wing/Glider/HELICOPTER
RF Range	2.408-2.475GHz
RF power	< 20dBm
RF Channel	135
Bandwidth	500KHz
2.4GHz System	AFHDS 2A / AFDHS
Modulation Type	GFSK
Stick Resolution	4096
Low Voltage Warning	< 4.2V
DSC port	PS/2 Port PPM
Chargeable	No
Antenna Length	26mm(Dual Antenna)
Weight	392 g
Power	6V DC 1.5AA*4
Display	STNTransflective Display ,LCD128x64 Lattice , VA 73x 39mm , LCD with white backlight
Size	174x89x190mm
On-line Update	Yes
Color	Black
Certificate	CE0678 , FCC ID:N4ZFLYSKYI6X

Récepteur

- alimenté par la batterie du robot et est équipé d'antennes pour recevoir les signaux radio.



Ghz Receiver (FS-iA6 (6 CH))

Channels	6
Model Type	Fixed-Wing/Glider/HElicopter
RF Range	2.408-2.475GHz
RF Channel	135
RF Receiver Sensitivity	- 105dBm
Bandwidth	500KHz
2.4GHz System	AFHDS 2A
Modulation Type	GFSK
Power	4.0~6.5V DC
Antenna Length	26mm(Dual Antenna)
Weight	7g
Size	40.4x21.1x15mm
i-BUS Port	No
Data Acquisition Port	No
Color	Black
Certificate	CE0678 , FCC ID:N4ZFLYSKYIA6

12.2 Mode Autonome (Navigation Assistée par Capteurs)

En mode **autonome**, le robot utilise ses capteurs LiDAR, caméra, et capteurs de pression pour naviguer **sans intervention humaine**.

Processus de commande en mode autonome :

- ***Acquisition des données environnementales:***

1. Le LiDAR détecte les obstacles et cartographie l'environnement en 3D.
2. La caméra capture des images pour analyse (détection d'anomalies).
3. Les capteurs de pression surveillent les vibrations ou anomalies mécaniques.

- ***Traitement des données :***

1. Le Raspberry Pi **analyse** les informations et détermine la meilleure trajectoire.
2. Il génère des commandes pour les moteurs (éviter un obstacle, ralentir, s'arrêter).

- ***Exécution des actions :***

1. L'ESP32 reçoit les ordres et ajuste les moteurs en conséquence.
2. Le robot ajuste sa direction et continue son inspection.

- ***Transmission des résultats :***

1. Les données de navigation sont envoyées **en temps réel** à l'opérateur via l'interface.
2. Un rapport d'inspection peut être généré automatiquement.

13. Communication entre les Modules

Composant	Connecté à	Protocole utilisé	Rôle
Raspberry Pi 5	ESP32	Wi-Fi / LoRa (SPI)	Envoi des commandes, collecte des données capteurs
Raspberry Pi 5	Interface utilisateur	WebSocket / Bluetooth / LoRa	Réception des commandes de l'opérateur
ESP32	Moteurs (PWN)	GPIO	Contrôle de la vitesse et direction
ESP32	Capteurs (LiDAR, DHT22, Pression)	UART / I2C / SPI	Acquisition des données environnementales
LiDAR	ESP32	I2C / SPI	Cartographie 3D et détection d'obstacles
Caméra	Raspberry Pi 5	CSI ou USB	Capture d'images pour inspection visuelle

Partie Dashboard InfluxDB de Surveillance des Anomalies des Trains

Introduction

Ce dashboard, construit avec **InfluxDB**, est un outil essentiel pour **surveiller** en temps réel divers paramètres critiques liés aux anomalies des trains. Il fournit une vue d'ensemble **des paramètres techniques** des trains, incluant des informations sur les bougies, **le groupe moteur réducteur**, **l'écoulement de l'huile**, et la **détection de vis non serrées**. Le tableau de bord suit également les anomalies et fournit des statistiques et des graphiques pour aider à analyser les données.

Fonctionnalités du Dashboard

1. **Total Inspection Time** : Un graphique qui affiche le temps total d'inspection écoulé.

1. **Inspection Speed** : Un panneau montre la vitesse de l'inspection en temps réel, permettant de mesurer l'efficacité des contrôles et la rapidité de l'analyse des anomalies.

1. **Anomaly Data Log** : Cette section présente un enregistrement détaillé des anomalies détectées, avec les informations de début et de fin ainsi que la valeur associée à chaque incident.

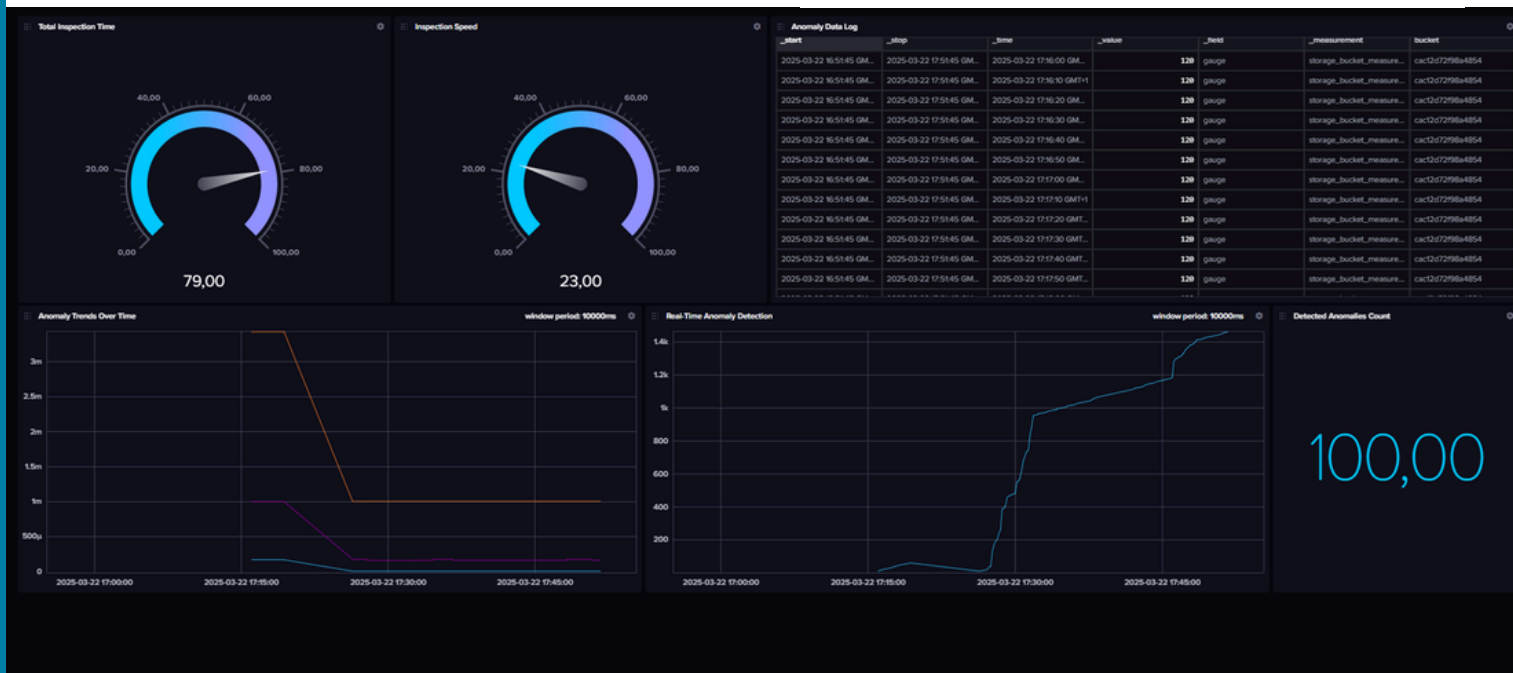
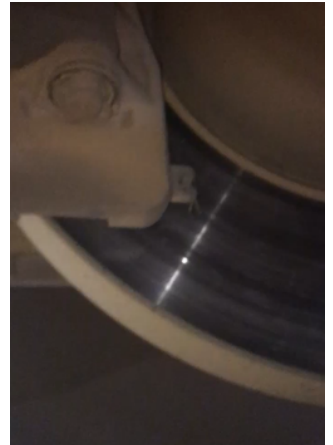
1. **Anomaly Trends Over Time** : Ce graphique montre l'évolution des anomalies détectées au fil du temps. Il permet d'identifier les périodes où les anomalies étaient les plus fréquentes et de détecter des tendances pouvant indiquer des problèmes récurrents.



1. Real-Time Anomaly Detection : Cette fonctionnalité permet de détecter des anomalies en temps réel.

Les données sont analysées instantanément

1. Detected Anomalies Count : Ce panneau affiche le nombre total d'anomalies détectées.



Fonctionnalités du Dashboard

1. Installation d'InfluxDB :

Télécharger et installer InfluxDB sur votre système. Vous pouvez obtenir le fichier d'installation depuis le site officiel d'InfluxDB. L'installation se fait en suivant les instructions adaptées à votre système d'exploitation (Linux, Windows, MacOS).

The following instructions include steps for downloading, verifying, and installing InfluxDB:

macOS

Linux

Windows

Docker

Kubernetes

Raspberry Pi

To install InfluxDB, do one of the following:

- [Install using Homebrew](#)
- [Manually download and install for macOS](#)



We recommend using [Homebrew](#) to install InfluxDB v2 on macOS.



InfluxDB and the influx CLI are separate packages

The InfluxDB server (`influxd`) and the `influx` CLI are packaged and versioned separately.

You'll install the `influx CLI` in a *later step*.

Install using Homebrew

```
brew update  
brew install influxdb
```

...

Install InfluxDB as a service with systemd

1. Choose the InfluxData key-pair for your OS version.
2. Run the command for your OS version to install the InfluxData key, add the InfluxData repository, and install `influxdb`.

Before running the command, replace the checksum and key filename with the key-pair from the preceding step.

```

# Ubuntu and Debian
# Add the InfluxData key to verify downloads and add the repository
curl --silent --location -O \
https://repos.influxdata.com/influxdata-archive.key
echo "943666881a1b8d9b849b74caebf02d3465d6beb716510d86a39f6c8e8dac7515  influxdata-
| sha256sum --check - && cat influxdata-archive.key \
| gpg --dearmor \
| sudo tee /etc/apt/trusted.gpg.d/influxdata-archive.gpg > /dev/null \
&& echo 'deb [signed-by=/etc/apt/trusted.gpg.d/influxdata-archive.gpg] https://repo
| sudo tee /etc/apt/sources.list.d/influxdata.list
# Install influxdb
sudo apt-get update && sudo apt-get install influxdb2
  
```

```

# RedHat and CentOS
# Add the InfluxData key to verify downloads
curl --silent --location -O \
https://repos.influxdata.com/influxdata-archive.key \
&& echo "943666881a1b8d9b849b74caebf02d3465d6beb716510d86a39f6c8e8dac7515  influxda
| sha256sum --check - && cat influxdata-archive.key \
| gpg --dearmor \
| tee /etc/pki/rpm-gpg/RPM-GPG-KEY-influxdata > /dev/null

# Add the InfluxData repository to the repository list.
cat <<EOF | tee /etc/yum.repos.d/influxdata.repo
[influxdata]
name = InfluxData Repository - Stable
baseurl = https://repos.influxdata.com/stable/${basearch}/main
enabled = 1
gpgcheck = 1
gpgkey = file:///etc/pki/rpm-gpg/RPM-GPG-KEY-influxdata
EOF
  
```

EOF

```
# Install influxdb  
sudo yum install influxdb2
```

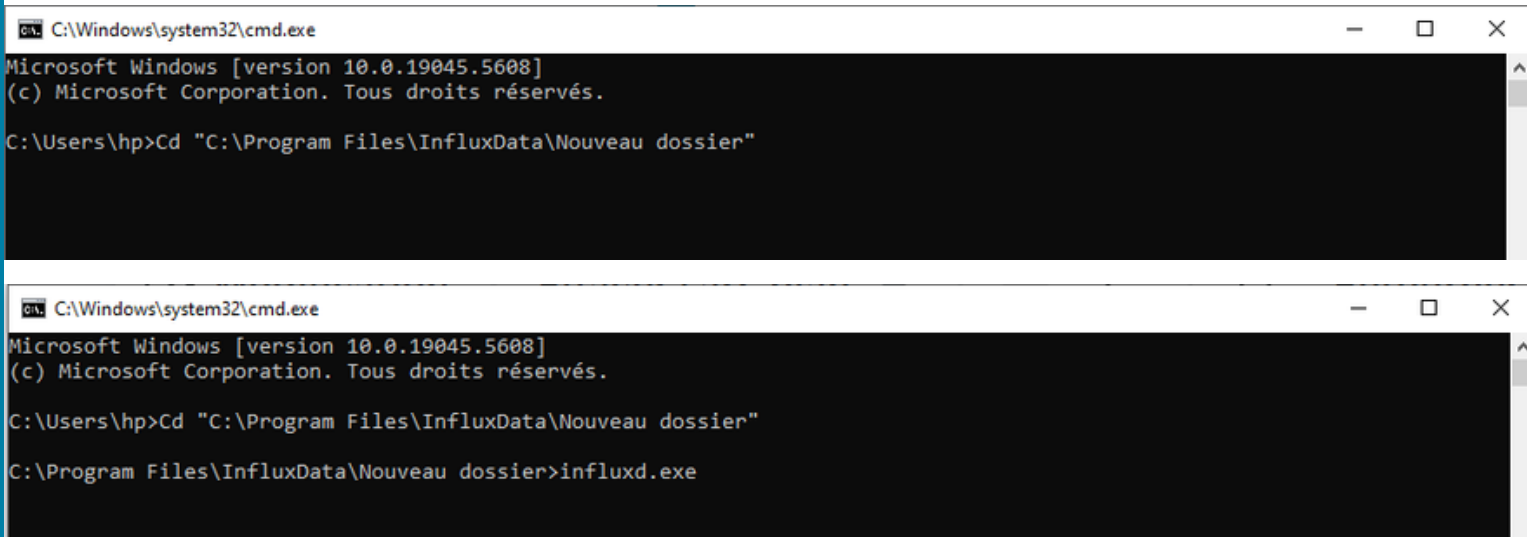
3. Start the InfluxDB service:

```
sudo service influxdb start
```

Installing the InfluxDB package creates a service file at `/lib/systemd/system/influxdb.service` to start InfluxDB as a background service on startup.

Pour plus d'information veuillez suivre ce lien :
<https://docs.influxdata.com/influxdb/v2/install/>

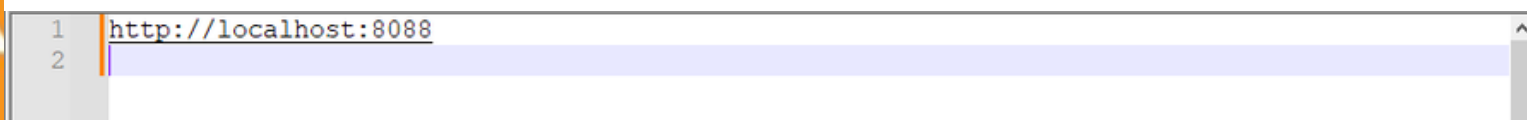
2. Exécution d'InfluxDB à partir du CMD (c'est important)



```
C:\Windows\system32\cmd.exe  
Microsoft Windows [version 10.0.19045.5608]  
(c) Microsoft Corporation. Tous droits réservés.  
C:\Users\hp>Cd "C:\Program Files\InfluxData\Nouveau dossier"  
  
C:\Windows\system32\cmd.exe  
Microsoft Windows [version 10.0.19045.5608]  
(c) Microsoft Corporation. Tous droits réservés.  
C:\Users\hp>Cd "C:\Program Files\InfluxData\Nouveau dossier"  
C:\Program Files\InfluxData\Nouveau dossier>influxd.exe
```

3. Accéder à l'Interface Web :

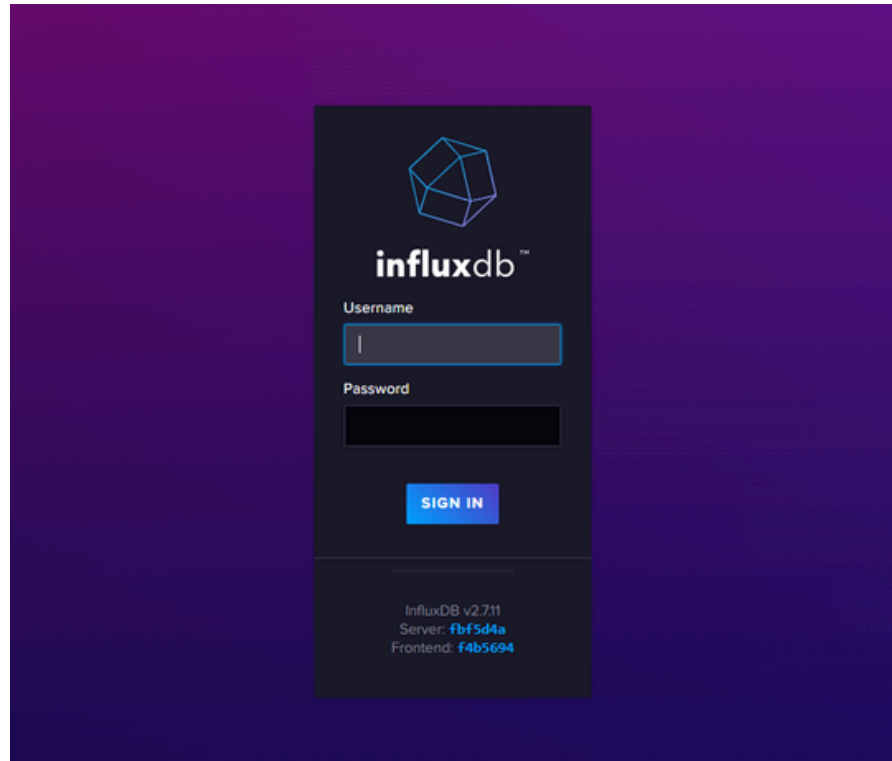
Pour accéder à l'interface web du dashboard en ouvrant votre moteur de recherche (Google, par exemple)



```
1 http://localhost:8088  
2
```

Username : Aymane

Password : AymaneAyoub2004@



InnovAM'25, conception d'un
robot d'inspection sous caisse,

Dimensionnement du moteur et de la batterie

**Étude importante, approfondie et bien
détaillée.**

Introduction

Dans un contexte où la maintenance prédictive et l'optimisation des processus industriels deviennent des enjeux majeurs, l'inspection automatisée des infrastructures ferroviaires représente une avancée stratégique. Ce projet, développé dans le cadre du concours **InnovAM'25**, propose la **conception d'un robot d'inspection sous caisse**, une solution innovante visant à améliorer l'efficacité, la sécurité et la précision des diagnostics dans l'entretien des trains.

L'objectif principal est de concevoir un système **autonome et intelligent** capable d'effectuer des inspections détaillées sous les véhicules ferroviaires, sans nécessiter l'intervention directe des techniciens en fosse. Grâce à une combinaison de **capteurs avancés, d'intelligence artificielle embarquée** et d'un châssis motorisé adapté aux environnements contraints, ce robot permettra de détecter de manière proactive les anomalies, réduisant ainsi les coûts de maintenance et augmentant la disponibilité des trains.

Ce rapport détaille les différentes étapes du projet, depuis l'analyse des contraintes techniques jusqu'à la conception mécanique, électronique et logicielle du robot, tout en intégrant les principes d'efficacité énergétique pour répondre aux exigences du concours InnovAM'25.

Chapter 1

Dimensionnement du moteur et de la batterie

1.1 Introduction

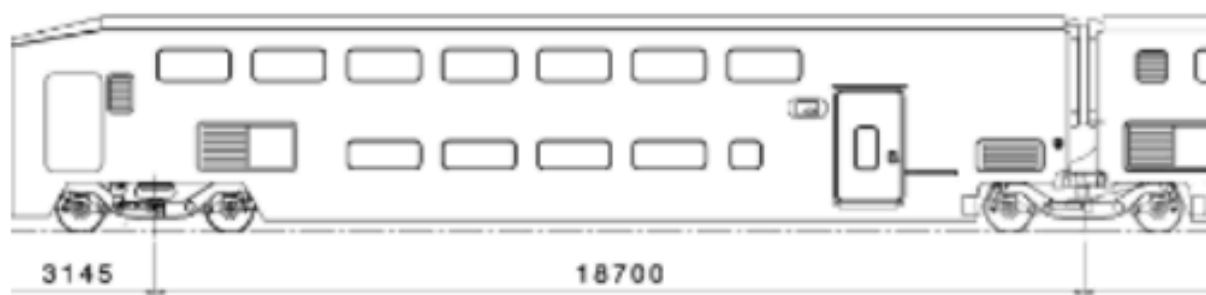
Le dimensionnement du moteur est une étape essentielle dans la conception du robot d'inspection sous caisse, influençant sa mobilité, son autonomie et ses performances. Ce chapitre détaille les critères de sélection, notamment le couple, la vitesse et l'efficacité énergétique, en tenant compte des contraintes mécaniques et environnementales. L'objectif est d'identifier la motorisation la plus adaptée pour assurer un déplacement fluide et fiable du robot.

1.2 Présentation du cycle moteur

1.2.1 Modélisation de la bogie pour l'inspection

On a choisi de modéliser la bogie sous forme de **carré 3m x 3m** pour simplifier le calcul de la distance parcourue et de la surface à inspecter. La forme carrée permet de rendre les calculs de déplacement plus intuitifs et de garantir que toutes les parties de la bogie sont couvertes de manière uniforme lors de l'inspection, sans omettre d'espaces importants. **La caméra de 1 mètre** de large a été choisie en fonction de la nécessité de capturer une portion significative de la bogie à chaque arrêt, tout en étant suffisamment large pour couvrir l'intégralité de la largeur du carré et s'assurer que l'inspection est efficace.

Concernant le raisonnement sur une seule rame intermédiaire de train contenant deux bogies, cela permet de réduire la complexité du modèle tout en conservant une représentation réaliste du système. Une rame intermédiaire typique est composée de deux bogies, ce qui permet de simuler le parcours et l'inspection d'une unité complète tout en restant fidèle à la configuration de la plupart des trains. Ce modèle simplifié permet de concentrer l'analyse sur les aspects essentiels du cycle de déplacement et d'inspection, tout en étant suffisamment représentatif pour en tirer des conclusions fiables sur la performance et les besoins en puissance du robot. **Il est important de noter que la puissance nécessaire pour l'inspection d'une rame de train reste relativement constante**, mais l'énergie consommée doit être multipliée par le nombre de rames à inspecter, ce qui influencera le dimensionnement de la batterie.



1.2.2 Phase d'Inspection

La phase d'inspection comprend des arrêts et des déplacements à des vitesses constantes, avec des phases d'accélération et de décélération. Les étapes de cette phase sont les suivantes :

1. Premier arrêt de 2 secondes :

- Avant toute action, un arrêt initial de 2 secondes est effectué.

2. Déplacement de 1,5 m :

- Accélération de 0 à 1,2 m/s en 0,6 s.
- Parcours de 1,5 m à une vitesse constante de 1,2 m/s pendant 1,25 s.
- Décélération de 1,2 m/s à 0 m/s en 0,6 s.
- Arrêt de 2 s.

3. Déplacement de 3 m :

- Accélération de 0 à 1,2 m/s en 0,125 s.

- Parcours de 3 m à une vitesse constante de **1,2 m/s** pendant **2,5 s**.
- Décélération de 1,2 m/s à 0 m/s en **0,125 s**.
- Arrêt de **2 s**.

4. Réitération des étapes 2 et 3 :

- Le cycle de déplacement de 1,5 m et de 3 m est répété une fois.

5. Accélération et phase stable :

- Accélération jusqu'à 1 m/s en **0,935 s**.
- Déplacement à une vitesse constante de **1 m/s** pendant **18,7 s**.
- Décélération de 1 m/s à 0 m/s en **0,935 s**.
- Arrêt de **2 s**.

1.2.3 Phase de déplacement entre Bogies

La phase suivante implique le déplacement entre les bogies, avec des accélérations et des décélérations spécifiques à chaque segment.

1. Accélération entre bogies :

- Accélération de 0 à 1 m/s en **0,935 s**.

2. Phase stable entre bogies :

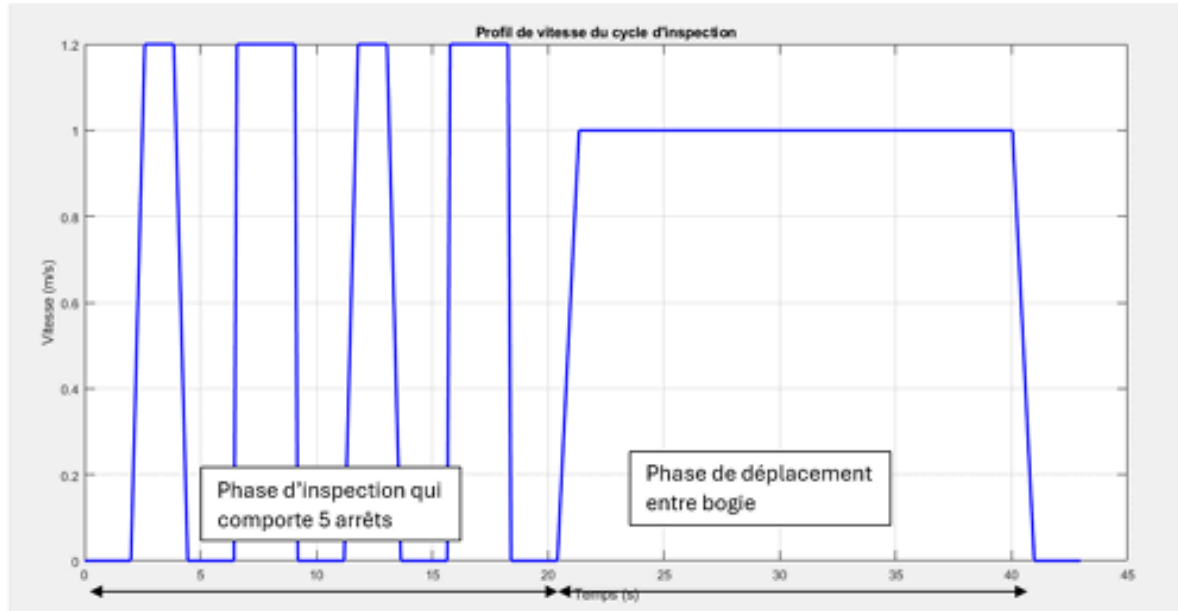
- Déplacement à une vitesse constante de **1 m/s** pendant **18,7 s**.

3. Décélération entre bogies :

- Décélération de 1 m/s à 0 m/s en **0,935 s**.

4. Arrêt final de 2 s :

- Un dernier arrêt de 2 s est effectué après la décélération.



1.3 Bilan des forces agissant sur le robot

L'expression de la force de roulement est donnée par :

$$F_{\text{roulage}} = \mu_{\text{roulage}} \cdot m \cdot g$$

où :

- F_{roulage} est la force de roulement,
- μ_{roulage} est le coefficient de frottement de roulement,
- m est la masse du véhicule,
- g est l'accélération gravitationnelle.

L'expression de la force du poids est donnée par :

$$F_{\text{Poids}} = m \cdot g \cdot \sin \alpha$$

où :

- F_{Poids} est la force du poids agissant sur un plan incliné,
- m est la masse de l'objet,
- g est l'accélération gravitationnelle,
- α est l'angle d'inclinaison du plan.

L'expression de la force aérodynamique est donnée par :

$$F_{\text{aéro}} = \frac{1}{2} \rho A C_d v^2$$

où :

- $F_{\text{aéro}}$ est la force de traînée aérodynamique,
- ρ est la densité de l'air,
- A est la surface frontale du véhicule,
- C_d est le coefficient de traînée,
- v est la vitesse du véhicule par rapport à l'air.

L'expression de la force d'accélération est donnée par :

$$F_{\text{acc}} = m \cdot a = m \cdot \frac{dv}{dt}$$

où :

- F_{acc} est la force d'accélération,
- m est la masse du véhicule,
- a est l'accélération,
- v est la vitesse du véhicule,
- $\frac{dv}{dt}$ représente la dérivée de la vitesse par rapport au temps.

L'expression du couple moteur est donnée par :

$$T_{mot} = \frac{F_{te} \cdot r}{G \cdot \eta}$$

où :

- T_{mot} est le couple moteur,
- F_{te} est la force exercée sur les roues motrices,
- r est le rayon de la roue,
- G est le rapport de transmission,
- η est le rendement de la transmission.

Or :

$$F_{te} = F_{roulage} + F_{Poids} + F_{aéro} + F_{acc}$$

L'expression complète est donnée par :

$$\eta \left(\frac{G}{r} \right) T_{mot} = (\mu_{roulage} + \sin \alpha) mg + 0.625 AC_d v^2 + 1.05 m \frac{dv}{dt}$$

En isolant $T_{\text{mot}}(\omega)$, on obtient :

$$T_{\text{mot}} = \frac{(\mu_{\text{roulage}} + \sin \alpha)mg + 0.625AC_d v^2 + 1.05m \frac{dv}{dt}}{\eta \cdot \left(\frac{G}{r}\right)}$$

1.4 Calcul de la puissance moyenne du moteur

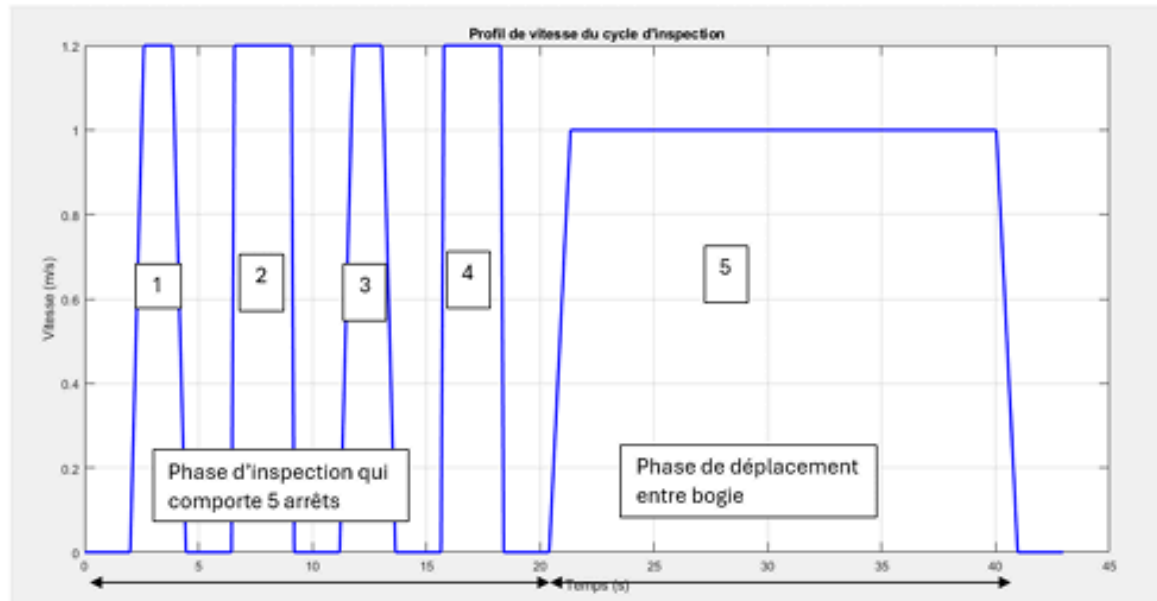
1.4.1 hypothèses

- $\mu_{\text{roulage}} = 0.015$
- Le coefficient de traînée est $C_d = 0.3$.
- La surface frontale est 0.015 m^2 .
- $G = 1: 2$ et $r=50\text{mm}$ ($G/r= 10$)
- $\eta = 0.9$
- $\alpha = 0$ (pas de pente $\sin(\alpha) = 0$)
- La masse est de 45 Kg

1.4.2 Calcul du couple moteur pour chaque phase :

- Pour la phase 1 (2.45 S) : $C_1 = 11.23 \text{ N.m}$
- Pour la phase 2 (2.75 s) : $C_2 = 51.13 \text{ N.m}$
- Pour la phase 3 (2.45 S) : $C_3 = 11.23 \text{ N.m}$
- Pour la phase 4 (2.75 s) : $C_4 = 51.13 \text{ N.m}$
- Pour la phase 5 (20.57 S) : $C_5 = 6.35 \text{ N.m}$

Sans oublier les phases d'arrêts de 2s pour un couple nulle.



1.4.3 Calcul du couple moteur moyen pondéré

$$C_{mi} = \sqrt{\frac{C_1^2 \cdot t_1 + C_2^2 \cdot t_2 + C_3^2 \cdot t_3 + \dots}{t_1 + t_2 + t_3 + \dots}}$$

ce qui donne : $C_{mi} = 19N.m$

1.4.4 Puissance moyenne du moteur requise

La vitesse de rotation maximal du moteur est calculé comme suit : $\Omega = V/R = 40rd/s$

La puissance requise est donc 786 W

D'où le choix d'un moteur avec une puissance nominal de 800 W disponible sous 24V, 36 V, 48V. on verra par la suite selon le dimensionnement de la batterie quelle tension sera la plus approprié.

1.4.5 Choix du moteur brushless DC

Critère	Moteur Brushless DC (BLDC)	Avantages
Efficacité énergétique	80-90%	Moins de pertes, meilleure autonomie
Durée de vie	Très longue (pas d'usure des balais)	Moins de maintenance
Maintenance	Faible	Pas de frottement mécanique interne
Densité de puissance	Élevée (puissant et compact)	Moins encombrant, meilleur rapport puissance/poids
Contrôle	Complexe (nécessite un contrôleur)	Très précis en vitesse et position
Couple à haute vitesse	Stable et performant	Idéal pour des accélérations rapides
Chauffe	Faible	Moins de pertes thermiques, meilleur rendement
Utilisation industrielle	Très répandu (robots, drones, véhicules électriques)	Fiabilité et haute performance

Table 1.1: Comparaison des performances d'un moteur Brushless DC (BLDC)

Les moteurs Brushless DC (BLDC) sont aujourd'hui largement adoptés par l'industrie pour leur rendement élevé, leur fiabilité et leur faible maintenance. Contrairement aux moteurs à balais, ils offrent une meilleure efficacité énergétique et une longévité accrue, ce qui les rend parfaits pour les applications industrielles comme les robots d'inspection, les drones et les véhicules électriques.

Dans le cas de notre robot d'inspection ferroviaire de 45 kg, le BLDC est particulièrement adapté car il permet :

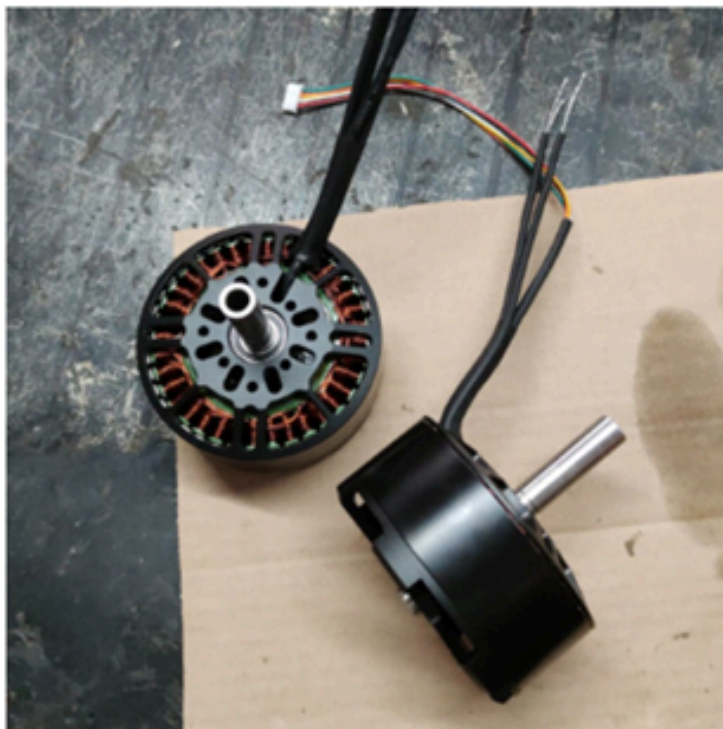
- Une puissance élevée avec un format compact pour s'intégrer sous les trains.
- Un faible échauffement, essentiel pour préserver l'électronique embar-

quée.

- Un couple stable à haute vitesse, garantissant des déplacements fluides et précis.
- Une autonomie optimisée, idéale pour un robot sur batterie.

En résumé, le moteur BLDC est un choix optimal, combinant performance, efficacité et durabilité, ce qui explique pourquoi il est devenu un standard industriel.

1.4.6 Présentation du moteur choisi et vérification

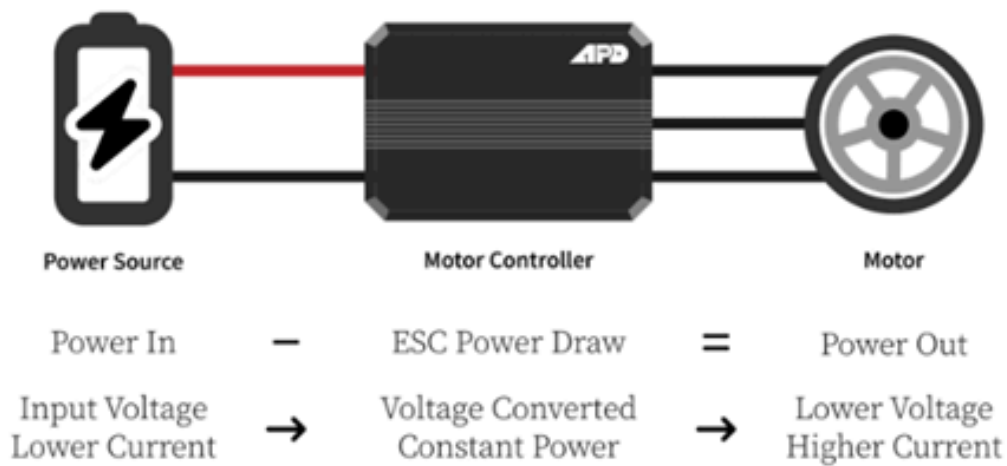


Category: 90mm
Construction: 36N/42P
Demension (mm): D92mm*47.5mm
Working Voltage: 50V
Max Torque (Nm): 9.5Nm
KV: 27KV, 1350rpm
Max Power (W): 800W
Rated Power: 600W
Max Continious current: 12A
Net Weight: 900g

 Inquiry

Ce moteur vérifie amplement les caractéristiques de couple et de vitesse recommandé.

Avec un couple de 9.5 N.m Avec un réducteur 1:2 qui va multiplier par 2 le couple au niveau de la sortie ce qui donne un couple de 19 Nm qui correspond au couple requis requis . **Ce type de moteur nécessite un contrôleur de vitesse qu'on appelle ESC Electronic Speed Controller. Ce contrôleur est compatible avec notre carte de rhasberry pi.**



1.5 Dimensionnement de batterie

1.5.1 le Bilan énergétique

Le dimensionnement de la batterie se fera pour une autonomie de 4h la durée d'un demi shift sousn 48V. L'utilisation d'une batterie Li-ion 48V pour un robot d'inspection de 45 kg présente plusieurs avantages significatifs. Tout d'abord, les batteries Li-ion offrent une densité énergétique élevée, ce qui permet de stocker plus d'énergie dans un volume et un poids réduits par rapport à d'autres technologies de batteries. Cela se traduit par une autonomie plus longue pour des performances élevées, tout en minimisant l'encombrement.

De plus, les batteries Li-ion sont réputées pour leur longue durée de vie, leur efficacité de charge et leur faible taux d'autodécharge, offrant ainsi une fiabilité accrue pour des applications industrielles. L'augmentation de poids liée à l'utilisation d'une batterie de 48V, bien que légèrement plus élevée, reste marginale dans le contexte global d'un robot de 45 kg, ce qui signifie que l'impact sur la mobilité et la maniabilité du robot est minime. Ainsi, choisir une batterie Li-ion 48V permet de concilier puissance, autonomie et durabilité, sans compromettre les performances globales du robot.

- Le moteur consomme 3200 Wh sous 48 V
- la carte Rhasberry Pi consomme 12 Wh
- le contrôleur de vitesse ESC 80 Wh
- Une caméra de 40 WH
- un éclairage de 1000 lumens 40 Wh
- un BMS de 20 Wh

L'énergie global requise pour un demi shift est de 3392 Wh.

La puissance global requise pour un demi shift est de 848 W.

Calcul du courant maximal fournit par la batterie :

$$I_{\max} = \frac{P_{\max}}{U_n} = 17.66 \text{ A}$$

La capacité de Peukert est calculée par la formule suivante :

$$C_{\text{Peukert}} = I_e^k \times t$$

Où :

$$- I_e = 17.66 \text{ A (courant à la puissance } k)$$

$$- k = 1.2$$

$$- t = 4 \text{ h}$$

Le calcul de la capacité de Peukert est donc :

$$C_{\text{Peukert}} = 17.66^{1.2} \times 4 = 125.5 \text{ Ah}$$

Nous devons dimensionner une batterie avec une tension nominale de 48 V et une capacité de 125 Ah. La capacité et la tension de chaque cellule lithium-ion (Li-ion) sont les suivantes :

$$- \text{Tension nominale d'une cellule Li-ion : } U_{\text{cellule}} = 3,7 \text{ V}$$

$$- \text{Capacité d'une cellule Li-ion : } C_{\text{cellule}} = 3,2 \text{ Ah}$$

Nous allons maintenant calculer le nombre de cellules nécessaires en série et en parallèle.

1.5.2 Calcul du nombre de cellules en série

Pour atteindre la tension nominale de 48 V, nous devons connecter plusieurs cellules en série. La tension totale d'une série de cellules est la somme des tensions des cellules individuelles. Le nombre de cellules en série est donné par :

$$N_{\text{cellules série}} = \frac{U_{\text{batterie}}}{U_{\text{cellule}}} = \frac{48 \text{ V}}{3,7 \text{ V/cellule}} \approx 13 \text{ cellules}$$

Ainsi, il nous faut ****13 cellules en série**** pour atteindre une tension de 48 V.

1.5.3 Calcul du nombre de cellules en parallèle

Maintenant, nous devons déterminer combien de cellules en parallèle sont nécessaires pour atteindre la capacité de 125.5 Ah. La capacité totale de la batterie est la somme des capacités des cellules en parallèle. Le nombre de cellules en parallèle est donné par :

$$N_{\text{cellules parallèle}} = \frac{C_{\text{batterie}}}{C_{\text{cellule}}} = \frac{125.5 \text{ Ah}}{3,2 \text{ Ah/cellule}} \approx 39.21$$

Comme le nombre de cellules en parallèle doit être un entier, nous arrondissons à 40 cellules en parallèle.

1.5.4 Nombre total de cellules

Le nombre total de cellules nécessaires pour construire la batterie est le produit du nombre de cellules en série et du nombre de cellules en parallèle. Ainsi, le nombre total de cellules est :

$$N_{\text{total}} = N_{\text{cellules série}} \times N_{\text{cellules parallèle}} = 13 \times 40 = 520 \text{ cellules}$$

1.5.5 Conclusion du dimensionnement de la batterie

Pour obtenir une batterie de 48 V et 161,77 Ah avec des cellules Li-ion de 3,7 V et 3,2 Ah, nous avons besoin de :

- 13 cellules en série pour atteindre la tension de 48 V,
- 51 cellules en parallèle pour atteindre la capacité de 125.25 Ah.

Le nombre total de cellules nécessaires est de 520 cellules. **Sachant** que chaque cellule a un poids de 48g alors la batterie a une masse de 25 Kg . Ce qui représente à peu près de 50% de la masse totale, ce qui est conforme aux normes pour ce genre d'application pour un robot d'inspection .

1.6 Freinage régénératif

Pour faire face à cette problématique on décide de mettre en place un système de freinage régénératif. Le freinage régénératif permet de récupérer une partie de l'énergie cinétique d'un système en mouvement. Cette énergie est donnée par la relation :

$$E_k = \frac{1}{2}mv^2 \quad (1.1)$$

où m est la masse du système et v sa vitesse. Cependant, cette énergie dépend du carré de la vitesse, ce qui signifie que pour des vitesses faibles, l'énergie récupérable devient insignifiante.

Dans le cas d'un robot d'inspection de masse $m = 45$ kg évoluant à une vitesse maximale de $v = 1.2$ m/s, l'énergie cinétique totale est :

$$E_k = \frac{1}{2} \times 45 \times (1.2)^2 = 32.4 \text{ J} \quad (1.2)$$

Avec un rendement de récupération d'environ 70%, l'énergie réellement récupérée par freinage est :

$$E_r = 0.7 \times 32.4 = 22.7 \text{ J} \quad (1.3)$$

Comparée à la puissance consommée moyenne du robot, qui est de 848 W, cette énergie est négligeable. En effet, en une seconde d'opération, le robot consomme :

$$E_{\text{consommée}} = 848 \text{ J/s} \quad (1.4)$$

ce qui est environ 37 fois plus élevé que l'énergie récupérée par freinage. De plus, sur une période de 4 heures avec environ 2058 freinages, l'énergie totale récupérée est :

$$E_{\text{totale}} = 2058 \times 22.7 = 46.6 \text{ kJ} = 12.95 \text{ Wh} \quad (1.5)$$

Ce qui représente une récupération énergétique de seulement :

$$\frac{12.95}{3392} \times 100 \approx 0.38\% \quad (1.6)$$

de l'énergie totale consommée. Cette valeur est extrêmement faible et n'apporte aucun gain significatif en autonomie.

Ainsi, le freinage régénératif est inefficace pour des véhicules de faible masse et faible vitesse.

1.7 Conclusion

Ce rapport a présenté l'étude et le dimensionnement du système de propulsion et d'alimentation d'un robot d'inspection évoluant sous caisse dans une fosse ferroviaire. L'analyse détaillée du cycle de vitesse a permis de définir avec précision les besoins énergétiques et de concevoir un moteur adapté aux sollicitations du cycle moteur.

Le dimensionnement de la batterie a été optimisé afin d'assurer une autonomie de 4 heures, tout en minimisant la masse globale du robot afin d'améliorer sa mobilité et son efficacité énergétique. L'étude du freinage régénératif a également été menée, mais son intégration n'a pas été retenue en raison de son faible impact énergétique, la vitesse du robot (1.2 m/s) ne permettant pas une récupération d'énergie significative.

Ainsi, le système conçu garantit un équilibre optimal entre performance, autonomie et maniabilité, répondant aux contraintes spécifiques de l'inspection ferroviaire en fosse. Des axes d'amélioration pourraient être envisagés pour affiner la gestion énergétique et optimiser davantage le rendement global du système.

Moteurs et actionneurs :

- **Moteurs pas à pas (pour le bras robotique)**
- **Moteurs DC (pour la propulsion)**
- **Servomoteurs (pour les ajustements du bras)**
- **Motoréducteurs (pour fournir un couple suffisant)**

Capteurs :

- **Caméra Raspberry Pi V3 : Pour la capture d'images haute résolution et l'inspection visuelle.**
- **LiDAR RPLIDAR A3M1 : Pour la cartographie 3D et la détection d'obstacles.**
- **Capteurs ultrasons SRF10 : Pour éviter les collisions avec les objets et le bras robotique.**
- **Capteur de température LM35 : Pour surveiller la température des moteurs et activer le refroidissement si nécessaire.**
- **Capteur infrarouge Sharp GP2Y0A02YK0F : Pour détecter la présence d'obstacles ou de personnes.**

Matériel mécanique :

- **Châssis en métal léger mais robuste pour supporter les composants électroniques et permettre un passage facile sous les trains.**
- **Bras robotique conçu pour des mouvements précis et sûrs.**

artes électroniques :

- **Raspberry Pi 5 (unité maître)**
- **ESP32 (unité esclave)**
- **Drivers de moteurs A4988 et L298N**

. Estimation du prix

L'estimation du prix pour la réalisation du robot se compose des coûts des différents composants, comme suit :

- Composants électroniques : Entre 400 et 500 € pour les cartes, capteurs, moteurs, et batteries.
- Matériel mécanique : Entre 300 et 400 € pour la fabrication du châssis et des pièces mécaniques nécessaires.
- Développement logiciel et intégration : Entre 200 et 300 € pour le développement des algorithmes d'intelligence artificielle, la gestion de la communication sans fil, et la programmation des systèmes embarqués.

Catégorie	Composant	Lien d'achat
Unité de traitement et contrôle	Raspberry Pi 5	https://www.robot.ma/raspberry-pi/
Unité de traitement et contrôle	ESP32	https://www.electrodoc.ma/microcontrôleurs/esp32
Capteurs et modules	Caméra Raspberry Pi V3	https://www.robot.ma/cameras-raspberry-pi/
Capteurs et modules	LiDAR RPLIDAR A3M1	https://www.robot.ma/lidar/
Capteurs et modules	IMU (accéléromètre + gyroscope + magnétomètre)	https://www.electrodoc.ma/capteurs-mouvement
Capteurs et modules	Capteur ultrasons SRF10	https://www.electrodoc.ma/capteurs-ultrasons
Capteurs et modules	Capteur de température LM35	https://www.electrodoc.ma/capteurs-temperature
Capteurs et modules	Capteur infrarouge Sharp GP2Y0A02YK0F	https://www.electrodoc.ma/capteurs-infrarouge
Capteurs et modules	Capteur de distance Sharp GP2Y0A02YK0F	https://www.electrodoc.ma/capteurs-infrarouge
Communication	Module LoRa RFM9xW433/868/915MHz	https://www.electrodoc.ma/modules-rf
Communication	Récepteur FS-ia6 6CH (2.4 GHz)	https://www.robot.ma/recepteurs-rc/
Moteurs et contrôle moteur	Stepper Motor	https://www.robot.ma/moteurs-pas-a-pas/
Moteurs et contrôle moteur	Motoreducteur	https://www.electrodoc.ma/motorisation/motoreducteurs
Moteurs et contrôle moteur	Driver moteur A4988	https://www.electrodoc.ma/motorisation/drivers-moteurs
Moteurs et contrôle moteur	Driver moteur L298N	https://www.robot.ma/drivers-moteurs/
Moteurs et contrôle moteur	Batterie LiPo 7.4V 2000mAh	https://www.robot.ma/batteries/
Alimentation	Batterie LiPo 12V	https://www.robot.ma/batteries/
Alimentation	Régulateur de tension 3.3V et 5V	https://www.electrodoc.ma/alimentation/regulateurs-tension
Alimentation	PCB personnalisé	À concevoir
Divers		

